The Language Astro



Astro is the first of five languages designed for a compiler course.

CONTENTS

1 Introduction • 2 Language Description • 3 Standard Library • 4 Formal Syntax • 5 Formal Semantics

1 Introduction

Astro is a fairly trivial programming language with interesting features that make it a great fit for introducing (1) compiler and interpreter writing, and (2) formal language semantics.

This document defines the language Astro.

2 Language Description

2.1 Programs

A **program** is a sequence of one or more statements. There are only two kinds of statements, assignments and print statements. Comments begin with `//` and extend to the end of the line.

```
// A simple program in Astro

radius = 55.2 * (-cos(2.8E-20) + 89) % 21;    // assignment statement
the_area = π * radius ** 2;                   // another assignment
print(hypot(2.28, 3 - radius) / the_area);    // print statement
```

Apologies for the semicolons, but they do make the language somewhat easier to parse.

2.2 Values and Types

All values in Astro are instances of a **type**. The language has the following types:

- The type `number` of IEEE-754 binary64 values.
- The types `function<`n`>` for all $n \geq 0$, representing functions from n numeric inputs to a single numeric output.

Numbers are first-class values, meaning they can be stored in variables, passed to functions, and returned from functions. Functions cannot: the only thing one can do with a function is call it.

Numbers are written as in JavaScript:

```
2
2.0
55.9
819.999e-15
2E+10
5.89999e2
```

2.3 Assignments

An **assignment** binds an entity to an identifier. Any identifier other than the built-in ones must be assigned to before it can subsequently be used.

```
sister = 5 + 1;              // variable declaration of sister (V + I = VI)
print(sister);               // okay, sister has been assigned
```

```
// print(cousin);          // ERROR: cousin has not been assigned
print(π);                  // Turns out to be okay because π is built-in
```

2.4 Functions

As Astro is a trivial language, you cannot define your own functions. You are limited to the built-in functions `sqrt`, `sin`, `cos`, and `hypot`. If a function is bound to an identifier, you cannot rebind the identifier:

```
print(sqrt(100));          // ok, sqrt are built-in
// sqrt = 5;               // ERROR: function already bound to sqrt
```

Functions can only be called. They cannot be used in a context where a number is expected:

```
// print(sin)              // ERROR
// t = sin;                // ERROR
// strange = sin * 3       // ERROR
```

Functions declared with n parameters must be passed exactly n arguments when called.

2.5 Variables

A **variable** is something that stores a value. Variables come into existence when assigned to for the first time. There is, however, one pre-declared variable, π. Variables can only be used if previously assigned or if built-in.

The variable `π` is read-only and all other variables are writable.

Variables can store numeric values only, not functions.

2.6 Statements

A **statement** is code that is executed solely for its side effect; it produces no value. The kinds of statements are:

- *i* = *e* ;

  (Assignment statement) *e* is evaluated, then the value of *e* is copied into *i*. *i* must not already have a function bound to it.

- **print** *e* ;

  (Print statement) Evaluate *e* then writes its evaluation to standard output.

2.7 Expressions

An **expression** produces a numeric. The Astro expressions are:

- A numeric literal.
- A variable occurrence, which must have been previously defined.
- – *e*

  Evaluates *e* and produces the negation of *e*.

- e_1 ** e_2

  The subexpressions are evaluated in any order and $e_1^{e_2}$ is produced.

- e_1 * e_2

  The subexpressions are evaluated in any order and their product is produced.

- e_1 / e_2

  The subexpressions are evaluated in any order and their quotient is produced.

- e_1 % e_2

  The subexpressions are evaluated in any order and the remainder of e_1 divided by e_2 is produced.

- e_1 + e_2

  The subexpressions are evaluated in any order and their sum is produced.

- e_1 – e_2

  The subexpressions are evaluated in any order and their difference is produced.

- $f(e_1, \ldots e_n)$

  (Function call) Evaluates each e_i then calls the function bound to f with these evaluated arguments, in order, and produces the returned value of the function. f must have a function of n parameters bound to it.

3 Standard Library

The following identifiers are pre-defined in a scope that surrounds the program. This means that none of these identifiers may be declared anywhere in a program.

- `π`

  Read-only variable whose value is the best approximate value of π.

- `function sqrt(x)`

  Returns the square root of x.

- `function sin(x)`

  Returns the sine of x radians.

- `function cos(x)`

  Returns the cosine of x radians.

- `function hypot(x, y)`

  Returns the hypotenuse of a right triangle with sides $|x|$ and $|y|$.

4 Formal Syntax

The source of a Astro program is a Unicode string. Here is the syntax given as an Ohm grammar:

astro.ohm

```
Astro {
  Program     = Statement+
```

```
  Statement   = id "=" Exp ";"                          --assignment
              | print Exp ";"                           --print
  Exp         = Exp ("+" | "-") Term                    --binary
              | Term
  Term        = Term ("*" | "/" | "%") Factor           --binary
              | Factor
  Factor      = Primary "**" Factor                     --binary
              | "-" Primary                             --negation
              | Primary
  Primary     = id "(" ListOf<Exp, ","> ")"             --call
              | numeral                                 --num
              | id                                      --id
              | "(" Exp ")"                             --parens

  numeral     = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  print       = "print" ~idchar
  idchar      = letter | digit | "_"
  id          = ~print letter idchar*
  space      += "//" (~"\n" any)*                       --comment
}
```

5 Formal Semantics

The meaning of an Astro program is defined in this section via transition rules in the style of Natural Semantics. It is defined from the following abstract syntax:

n : Nml		
i : Ide		
e : Exp	$=$	$n \mid i \mid -e \mid e+e \mid e-e \mid e*e \mid e/e \mid e\%e \mid e**e \mid \textbf{call}\ i\ e^*$
s : Stm	$=$	$i = e \mid \textbf{print}\ e$
p : Pro	$=$	$\textbf{program}\ s^+$

The meaning of a Astro program at runtime is the list of values it prints. To formally specify this behavior, we also have to define the meanings of statements and expressions. We do this with the help of a **memory**, which maps identifiers to their runtime values, and the **output**, which is the list of values output so far. The type **Value** is defined as:

$$\frac{}{\mathsf{Undef} : \mathsf{Value}} \qquad \frac{x : \mathsf{Real} \quad b : \mathsf{Bool}}{\mathsf{Num}\ x\ b : \mathsf{Value}} \qquad \frac{f : \mathsf{Real}^* \to \mathsf{Real} \quad n : \mathsf{Nat}}{\mathsf{Fun}\ f\ n : \mathsf{Value}}$$

allowing identifiers to be (1) bound to a variable with a mutability flag, (2) bound to a function together with its parameter count (so that the number of arguments can be checked at call time), or (3) not yet defined. The predefined types **Bool**, **Nat**, and **Real** refer to booleans, natural numbers, and IEEE-754 binary64 values, respectively. Each statement is executed in the context of a **state**, which is the current memory together with the output so far. Expressions need only be evaluated in the context of the current memory, as they do not read nor modify the output. The semantic rules are:

$$\frac{}{m \vdash [\![n]\!] \Downarrow n} \qquad \frac{m(i) = \mathsf{Num}\ x\ b}{m \vdash [\![i]\!] \Downarrow x} \qquad \frac{m \vdash e \Downarrow x}{m \vdash [\![- e]\!] \Downarrow -x}$$

$$\frac{op \in \{+, -, *, /, \%, **\} \quad m \vdash e_1 \Downarrow x \quad m \vdash e_2 \Downarrow y}{m \vdash [\![e_1\ op\ e_2]\!] \Downarrow op(x, y)} \qquad \frac{(m \vdash e_i \Downarrow a_i)_{i=1}^{n} \quad m(i) = \mathsf{Fun}\ f\ n}{m \vdash [\![\mathtt{call}\ i\ e_1, \ldots, e_n]\!] \Downarrow f(a_1, \ldots, a_n)}$$

$$\frac{m \vdash e \Downarrow x \quad m(i) = \mathsf{Undef} \vee m(i) = \mathsf{Num}\ y\ \mathsf{true}}{(m, o) \vdash [\![i = e]\!] \Downarrow (m[i \mapsto \mathsf{Num}\ x\ \mathsf{true}], o)} \qquad \frac{m \vdash e \Downarrow x}{(m, o) \vdash [\![\mathtt{print}\ e]\!] \Downarrow (m, o \cdot x)}$$

$$\frac{((m_{i-1}, o_{i-1}) \vdash s_i \Downarrow (m_i, o_i))_{i=1}^{n}}{\vdash [\![\mathtt{program}\ s_1, \ldots, s_n]\!] \Downarrow o_n}$$

where o_0, the initial output, is defined to be the empty sequence, and the initial memory m_0 is our "standard library" defined as follows:

$$\begin{aligned}
m_0 = (&\lambda i.\ \mathsf{Undef})[\\
&\pi \mapsto \mathsf{Num}\ \pi\ \mathsf{false}][\\
&\mathtt{sqrt} \mapsto \mathsf{Fun}\ (\lambda x.\ \sqrt{x})\ 1][\\
&\mathtt{sin} \mapsto \mathsf{Fun}\ (\lambda x.\ \sin x)\ 1][\\
&\mathtt{cos} \mapsto \mathsf{Fun}\ (\lambda x.\ \cos x)\ 1][\\
&\mathtt{hypot} \mapsto \mathsf{Fun}\ (\lambda (x, y).\ \sqrt{x^2 + y^2})\ 2]
\end{aligned}$$

The Language Bella

Bella is the second of five languages designed for a compiler course.

CONTENTS

1 Introduction • 2 Language Description • 3 Standard Library • 4 Formal Syntax • 5 Formal Semantics

1 Introduction

Bella is a simple programming language with interesting features that make it a great fit for learning (1) compiler and interpreter writing, and (2) formal language semantics.

The language is limited to numbers, assignments, conditionals, loops, and simple functions. Variables can only hold numbers, not functions. Function "bodies" are limited to simple expressions (like Python lambdas), so we don't have to deal with any scoping complications like temporal dead zones. Scoping is as in Python: local variables and parameters are function-scoped, and shadowing is allowed. Bella is a friendly, simple language, relatively easy to implement.

This document defines the language Bella.



2 Language Description

2.1 Programs

A **program** is a sequence of one or more statements.

```
let dozen = 12;                                    // variable declaration
print dozen % 3 ** 1;                              // print statement
function gcd(x, y) = y == 0 ? x : gcd(y, x % y);   // function declaration
while dozen >= 3 || (gcd(1, 10) != 5) {            // while statement
  dozen = dozen - 2.75E+19 ** 1 ** 3;              // assignment statement
}
```

Apologies for the semicolons, but they do make the language somewhat easier to parse.

2.2 Values and Types

All values in Bella are instances of a **type**. The language has the following types:

- The type `number` of IEEE-754 binary64 values.
- The types `function<`n`>` for all $n \geq 0$, representing functions from n numeric inputs to a single numeric output.

Numbers are first-class values, meaning they can be stored in variables, passed to functions, and returned from functions. Functions cannot be: the only thing one can do with a function is call it.

Numbers are written as in JavaScript:

```
2
2.0
55.9
819.999e-15
2E+10
5.89999e2
```

2.3 Declarations

A **declaration** binds an identifier to an entity. There are three kinds of declarations:

- function declarations
- variable declarations

- parameter declarations

Here is a short example showing all three kinds:

```
let sister = 5 + 1;           // variable declaration of sister (V + I = VI)
function triple(x) = x * 3;   // function declaration of triple
                              // ...and parameter declaration of x
```

Each occurrence of an identifier is either a **defining** occurrence or a **using** occurrence. Using occurrences are legal only in the **visible region** of the declaration that declares the identifier. In Bella, **shadowing** of top-level variables by parameters of local functions is permitted, so the visible region of a declaration is the scope minus any "inner" scopes of declarations of identifiers with the same name and may therefore be discontinuous.

- The scope of an identifier declared in a parameter declaration is the body of the innermost function in which the parameter declaration appears.
- The scope of an identifier declared in a variable declaration begins immediately after its initializer and extends to the end of the program.
- The scope of an identifier declared in a function declaration begins immediately after its parameter list and extends to the end of the program.

The identifiers appearing as defining occurrences of variable and function declarations must be mutually exclusive. A function's parameters must be mutually exclusive as well:

```
let x = 3;
// let x = 5;              // ERROR: x already declared
// function x() = 0;       // ERROR: x already declared
// function g(a, a) = 0;   // ERROR: a already declared
function h(x) = 0;         // OK! parameter x shadows global x
// let h = 3;              // ERROR: h already declared (prev line)
function j(k) = π - x;     // OK
let k = 2;                 // OK! Parameter k on prev line not in scope
```

2.4 Functions

As a simple language, Bella does not have any anonymous functions; all functions must be named. The identifier used in the function declaration is read-only variable: you cannot reassign it:

```
function successor(n) = n + 1;
// let successor = 5;            // ERROR: read only
```

Functions can only be called. They cannot be used in a context where a number is expected:

```
function triple (x) = x * 3;
print triple(8);                 // Function call of triple, perfectly fine
// print triple                  // ERROR
// let t = triple;               // ERROR
// let times_nine = triple * 3   // ERROR
```

Functions declared with n parameters must be passed exactly n arguments when called.

2.5 Variables

A **variable** is something that stores a value. Variables come into existence either (1) as regular variables declared in a variable declaration statement (using `let`), or (2) as parameters in a parameter declaration. There is one pre-declared variable, π. Variables can only be used if previously declared.

The variable `π` is read-only and all other variables are writable. Interestingly, because parameters are scoped only to the function body and function bodies are simple expressions, all parameters are effectively read-only.

Variables can store numeric values only, not functions.

2.6 Statements

A **statement** is code that is executed solely for its side effect; it produces no value. The kinds of statements are:

- `let` i `=` e `;`

(Variable declaration statement) Declares a new variable with name i. This declaration must be unique within its scope. Evaluates e and assigns this value to the new variable. The new variable is writable.

- `function` f `(` x_1`,...,`x_n `)` `=` e `;`

  (Function declaration) Defines a new function named f with n parameters, that is, its type is `function<`n`>`. The identifier f must be unique within its scope. The parameters must be unique within their scope.

- i `=` e `;`

  (Assignment statement) e is evaluated, then the value of e is copied into i. i must have been previously declared and visible in this scope.

- `print` e `;`

  (Print statement) Evaluates e then prints its value to standard output.

- `while` e b

  (While statement) First, e is evaluated. If e produces 0, the execution of the while statement terminates. Otherwise, body b is executed then the entire while statement is executed again.

2.7 Expressions

An **expression** produces a numeric. The Bella expressions are:

- A numeric literal.
- `true`

  Produces 1.

- `false`

  Produces 0.

- A variable occurrence, which must have been previously declared and visible in this scope.
- `-` e

  Evaluates e and produces the negation of e.

- `!` e

If e evaluates to 0, produces 1, otherwise produces 0.

- e_1 `*` e_2

  The subexpressions are evaluated in any order and their product is produced.

- e_1 `/` e_2

  The subexpressions are evaluated in any order and their quotient is produced.

- e_1 `%` e_2

  The subexpressions are evaluated in any order and the remainder of e_1 divided by e_2 is produced.

- e_1 `+` e_2

  The subexpressions are evaluated in any order and their sum is produced.

- e_1 `-` e_2

  The subexpressions are evaluated in any order and their difference is produced.

- e `?` e_1 `:` e_2

  Evaluates e and if non-zero, evaluates and produces e_1. Otherwise evaluates and produces e_2.

- e_1 `<` e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is less than the value of e_2.

- e_1 `<=` e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is less or equal to the value of e_2.

- e_1 `==` e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is equal to the value of e_2.

- e_1 `!=` e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is not equal to the value of e_2.

- e_1 >= e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is greater or equal to the value of e_2.

- e_1 > e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is greater than the value of e_2.

- e_1 && e_2

  First e_1 is evaluated. If it evaluates to 0, the entire expression immediately produces 0 (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

- e_1 || e_2

  First e_1 is evaluated. If it evaluates to a non-zero value, the entire expression immediately produces this value (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

3 Standard Library

The following identifiers are pre-defined in a scope that surrounds the program. This means that none of these identifiers may be declared anywhere in a program.

- π

  Read-only variable whose value is the best approximate value of π.

- `function sqrt(x)`

  Returns the square root of x.

- `function sin(x)`

  Returns the sine of x radians.

- `function cos(x)`

  Returns the cosine of x radians.

- `function exp(x)`

  Returns e^x.

- `function ln(x)`

  Returns the natural log of x.

- `function hypot(x, y)`

  Returns the hypotenuse of a right triangle with sides $|x|$ and $|y|$.

4 Formal Syntax

The source of a Bella program is a Unicode string. Here is the syntax given as an Ohm grammar:

bella.ohm

```
Bella {
  Program   = Statement+
  Statement = let id "=" Exp ";"                          -- vardec
            | function id Params "=" Exp ";"              -- fundec
            | Exp7_id "=" Exp ";"                         -- assign
            | print Exp ";"                               -- print
            | while Exp Block                             -- while
  Params    = "(" ListOf<id, ","> ")"
  Block     = "{" Statement* "}"

  Exp       = ("-" | "!") Exp7                            -- unary
            | Exp1 "?" Exp1 ":" Exp                       -- ternary
            | Exp1
  Exp1      = Exp1 "||" Exp2                              -- binary
            | Exp2
  Exp2      = Exp2 "&&" Exp3                              -- binary
            | Exp3
  Exp3      = Exp4 ("<="|"<"|"=="|"!="|">="|">") Exp4     -- binary
            | Exp4
  Exp4      = Exp4 ("+" | "-") Exp5                       -- binary
            | Exp5
```

```
  Exp5      = Exp5 ("*" | "/" | "%") Exp6                -- binary
            | Exp6
  Exp6      = Exp7 "**" Exp6                             -- binary
            | Exp7
  Exp7      = num
            | true
            | false
            | id "(" ListOf<Exp, ","> ")"                -- call
            | id                                         -- id
            | "(" Exp ")"                                -- parens

  let       = "let" ~idchar
  function  = "function" ~idchar
  while     = "while" ~idchar
  true      = "true" ~idchar
  false     = "false" ~idchar
  print     = "print" ~idchar
  keyword   = let | function | while | true | false
  num       = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  id        = ~keyword letter idchar*
  idchar    = letter | digit | "_"
  space    += "//" (~"\n" any)*                          -- comment
}
```

5 Formal Semantics

The meaning of a Bella program is defined in this section via transition rules in the style of Natural Semantics. It is defined from the following abstract syntax:

n: Numeral
i: Identifier
e: Expression $= n \mid i \mid \texttt{true} \mid \texttt{false} \mid \mathit{unaryop}\ e \mid e_1\ \mathit{binop}\ e_2 \mid \texttt{call}\ i\ e^* \mid e\ \texttt{?}\ e_1\ \texttt{:}\ e_2$
s: Statement $= \texttt{let}\ i = e \mid \texttt{func}\ i\ i^* = e \mid i = e \mid \texttt{print}\ e \mid \texttt{while}\ e\ b$
b: Block $= \texttt{block}\ s^*$
p: Program $= \texttt{program}\ b$

The unary operators are `-` and `!`. The binary operators are `+`, `-`, `*`, `/`, `%`, `**`, `<`, `<=`, `==`, `!=`, `>=`, `>`, `&&`, and `||`.

The meaning of a Astro program at runtime is the list of values it prints. To formally specify this behavior, we also have to define the meanings of statements and expressions. We do this with the help of a **memory**, which maps identifiers to their runtime values, and the **output**, which is the list of values output so far. The type **Value** is defined as:

$$\frac{}{\textsf{Undef}\colon \textsf{Value}} \qquad \frac{x\colon \textsf{Real} \quad b\colon \textsf{Bool}}{\textsf{Num}\ x\ b\colon \textsf{Value}} \qquad \frac{p\colon \textsf{Ide}^* \quad e\colon \textsf{Exp}}{\textsf{Fun}\ p\ e\colon \textsf{Value}} \qquad \frac{f\colon \textsf{Real}^* \to \textsf{Real} \quad n\colon \textsf{Nat}}{\textsf{StdFun}\ f\ n\colon \textsf{Value}}$$

allowing identifiers to be (1) bound to a variable with a mutability flag, (2) bound to a (built-in, or "standard") function together with its parameter count (so that the number of arguments can be checked at call time), (3) a user-defined function with a parameter list and a body expression, or (4) not yet defined. The predefined types **Bool**, **Nat**, and **Real** refer to booleans, natural numbers, and IEEE-754 binary64 values, respectively. Each statement is executed in the context of a **state**, which is the current memory together with the output so far. Expressions need only be evaluated in the context of the current memory, as they do not read nor modify the output. The semantic rules are:

$$\frac{}{m \vdash [\![n]\!] \Downarrow n} \qquad \frac{}{m \vdash [\![\texttt{true}]\!] \Downarrow 1} \qquad \frac{}{m \vdash [\![\texttt{false}]\!] \Downarrow 0}$$

$$\frac{m(i) = \textsf{Num}\ x\ b}{m \vdash [\![i]\!] \Downarrow x} \qquad \frac{m \vdash e \Downarrow x}{m \vdash [\![- e]\!] \Downarrow -x} \qquad \frac{m \vdash e \Downarrow x \quad x \neq 0}{m \vdash [\![\texttt{!}\ e]\!] \Downarrow 0}$$

$$\frac{m \vdash e \Downarrow 0}{m \vdash [\![\texttt{!}\ e]\!] \Downarrow 1} \qquad \frac{op \in \{+, -, *, /, \%, **\} \quad m \vdash e_1 \Downarrow x \quad m \vdash e_2 \Downarrow y}{m \vdash [\![e_1\ op\ e_2]\!] \Downarrow op(x, y)}$$

$$\frac{op \in \{<, <=, ==, \texttt{!}=, >=, >\} \quad m \vdash e_1 \Downarrow x \quad m \vdash e_2 \Downarrow y \quad op(x, y)}{m \vdash [\![e_1\ op\ e_2]\!] \Downarrow 1} \qquad \frac{op \in \{<, <=, ==, \texttt{!}=, >=, >\} \quad m \vdash e_1 \Downarrow x \quad m \vdash e_2 \Downarrow y \quad \neg op(x, y)}{m \vdash [\![e_1\ op\ e_2]\!] \Downarrow 0}$$

$$\frac{m \vdash e_1 \Downarrow 0}{m \vdash [\![e_1\ \&\&\ e_2]\!] \Downarrow 0} \qquad \frac{m \vdash e_1 \Downarrow x \quad x \neq 0 \quad m \vdash e_2 \Downarrow y}{m \vdash [\![e_1\ \&\&\ e_2]\!] \Downarrow y}$$

$$\frac{m \vdash e_1 \Downarrow x \quad x \neq 0}{m \vdash [\![e_1\ ||\ e_2]\!] \Downarrow x} \qquad \frac{m \vdash e_1 \Downarrow 0 \quad m \vdash e_2 \Downarrow y}{m \vdash [\![e_1\ ||\ e_2]\!] \Downarrow y}$$

$$\frac{m \vdash e \Downarrow x \quad x \neq 0 \quad m \vdash e_1 \Downarrow y}{m \vdash [\![e \texttt{ ? } e_1 \texttt{ : } e_2]\!] \Downarrow y} \qquad \frac{m \vdash e \Downarrow 0 \quad m \vdash e_2 \Downarrow z}{m \vdash [\![e \texttt{ ? } e_1 \texttt{ : } e_2]\!] \Downarrow z}$$

$$\frac{(m \vdash e_i \Downarrow a_i)_{i=1}^n \quad m(i) = \mathsf{Fun}\ (p_1, \ldots, p_n)\ e' \quad m[p_i \mapsto a_i]_{i=1}^n \vdash e' \Downarrow x}{m \vdash [\![\texttt{call } i\ e_1, \ldots, e_n]\!] \Downarrow x}$$

$$\frac{(m \vdash e_i \Downarrow a_i)_{i=1}^n \quad m(i) = \mathsf{StdFun}\ f\ n}{m \vdash [\![\texttt{call } i\ e_1, \ldots, e_n]\!] \Downarrow f(a_1, \ldots, a_n)}$$

$$\frac{m \vdash e \Downarrow x \quad m(i) = \mathsf{Undef}}{(m, o) \vdash [\![\texttt{let } i = e]\!] \Downarrow (m[i \mapsto \mathsf{Num}\ x\ \mathsf{true}], o)}$$

$$\frac{m(i) = \mathsf{Undef}}{(m, o) \vdash [\![\texttt{fun } i\ (p_1, \ldots, p_n) = e]\!] \Downarrow (m[i \mapsto \mathsf{Fun}\ (p_1, \ldots, p_n)\ e], o)}$$

$$\frac{e, m \Downarrow x \quad m(i) = (\mathsf{Num}\ y\ \mathsf{true})}{(m, o) \vdash [\![i = e]\!] \Downarrow (m[i \mapsto (\mathsf{Num}\ x\ \mathsf{true})], o)} \qquad \frac{m \vdash e \Downarrow x}{(m, o) \vdash [\![\texttt{print } e]\!] \Downarrow (m, o \cdot x)}$$

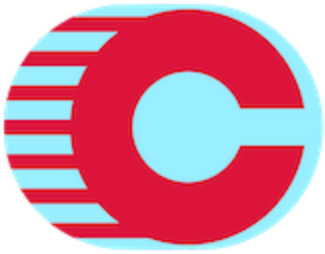
$$\frac{m \vdash e \Downarrow 0}{(m, o) \vdash [\![\texttt{while } e\ b]\!] \Downarrow (m, o)} \qquad \frac{\begin{array}{c} m \vdash e \Downarrow x \quad x \neq 0 \quad (m, o) \vdash b \Downarrow (m', o') \\ (m', o') \vdash [\![\texttt{while } e\ b]\!] \Downarrow (m'', o'') \end{array}}{(m, o) \vdash [\![\texttt{while } e\ b]\!] \Downarrow (m'', o'')}$$

$$\frac{((m_i, o_i) \vdash s_i \Downarrow (m_{i+1}, o_{i+1}))_{i=1}^n}{(m_1, o_1) \vdash [\![\texttt{block } s_1, \ldots, s_n]\!] \Downarrow o_{n+1}} \qquad \frac{(m_0, o_0) \vdash b \Downarrow (m, o)}{[\![\texttt{program } b]\!] \Downarrow o}$$

where o_0, the initial output, is defined to be the empty sequence, and the initial memory m_0 is our "standard library" defined as follows:

$$\begin{aligned}
m_0 = (&\lambda i.\ \mathsf{Undef})[\\
&\pi \mapsto \mathsf{Num}\ \pi\ \mathsf{false}][\\
&\texttt{sqrt} \mapsto \mathsf{Fun}\ (\lambda x.\ \sqrt{x})\ 1][\\
&\texttt{sin} \mapsto \mathsf{Fun}\ (\lambda x.\ \sin x)\ 1][\\
&\texttt{cos} \mapsto \mathsf{Fun}\ (\lambda x.\ \cos x)\ 1][\\
&\texttt{exp} \mapsto \mathsf{Fun}\ (\lambda x.\ e^x)\ 1][\\
&\texttt{ln} \mapsto \mathsf{Fun}\ (\lambda x.\ \ln x)\ 1][\\
&\texttt{hypot} \mapsto \mathsf{Fun}\ (\lambda (x, y).\ \sqrt{x^2 + y^2})\ 2]
\end{aligned}$$

The Language Carlos



The Carlos language was created sometime in the 1990s for a compiler construction class and has evolved a bit since. It is meant to be small, but not trivial, with features interesting enough to make writing a compiler an educational experience, but extremely clean so writing a compiler does not require handling difficult-to-implement tricky cases.

CONTENTS

1 Introduction • 2 Semantics • 3 Syntax • 4 Standard Library

1 Introduction

Carlos is a statically-typed, block-structured programming language with first-class functions. It has no explicit pointers, so assumes the existence of a garbage collector. While several types do have reference semantics, there is no billion-dollar mistake, since the null value is not a member of all reference types, but only of optional types. The language is rather small, with arrays, simple structs, and functions.

```
const languageName = "Carlos";

function greeting(): string {
  return random(["Welcome", "こんにちは", "Bienvenido"]);
}

print("👋👋👋");
repeat 5 {
```

```
  print(greeting() + " " + languageName);
}
```

This document defines the language Carlos.

2 Semantics

2.1 Programs

A **program** is a sequence of one or more statements.

```
// This program prints "hello, world" to standard output.

let greeting = "hello";             // variable declaration
function place(): string {          // function declaration
  return "world";
}
print(greeting + ", " + place());   // function call statement
```

Apologies for the semicolons, but they do make the language somewhat easier to parse.

2.2 Values and Types

All values in Carlos are instances of a type. The language has the following types:

- the type `boolean` consisting entirely of the values `true` and `false`.
- the type `int` of (unbounded) integers.
- the type `float` of IEEE-754 binary64 values.
- the type `string` of character strings.
- the type `void` which has no instances at all.
- the type `any` which is the union of all types.
- *array types*, written `[T]`, whose instances are zero-based integer-indexed sequences of values of type `T`.

- *structure types*, whose instances are ordered sequence of named *fields*.
- *optional types*, written `T?`, representing "boxes" that either contain a value of type `T` or don't contain anything.
- *function types*, written `(T1,T2,...,Tn)->T0`, consisting of all functions mapping (ordered) parameters of types `T1` through `Tn` to a value of type `T0`.

Booleans, integers, floats, and strings have literal forms:

```
true                                // has type boolean
false                               // has type boolean
2                                   // has type int
8912313125712589125870719 2         // has type int
2.0                                 // has type float
55.9                                // has type float
819.999E-15                         // has type float
"¡hola!"                            // has type string
"π™\t\n \" \' \u{1f4a9}"            // has type string
"I'm 💀"                            // has type string
```

The values in an array expression must all have exactly the same type; it is a compile-time error if they do not. Empty arrays *must* mention the intended type of the elements:

```
[true, true, false]                 // has type [boolean]
[3, 0, 5, 99 ** 250]                // has type [int]
[[1, 4], [2, 3]]                    // has type [[int]]
// [3, 0, 2.0]                      // ERROR Cannot mix ints and floats
// [3, "3"]                         // ERROR: Cannot mix ints and strings
[string]()                          // empty array of type [string]
// []                               // Syntax error, cannot infer a type
```

For optionals, use the `some` operator to create an optional with a present value. The operator `no` makes an empty optional on the given type.

```
some 5                              // has type int?
some ["red", "green", "blue"]       // has type [string]?
no boolean                          // has type boolean?
```

There are no literals for functions; they have to be named:

```
function sameMagnitude(x: int, y: int): boolean {
  return x == y || x == -y;
}

function printSecond(messages: [string]) {
  print(messages[1]);
}

function twice(f: (float)->float, x: float): float {
  return f(f(x));
}

sameMagnitude                       // has type (int, int)->boolean
printSecond                         // has type ([string])->void
twice                               // has type ((float)->float, float)->float
```

There are no type expressions for struct types; structure types have to be declared and referred to only by name. Instances of struct types are created using the struct name as a constructor:

```
struct Vector {i: float j: float}   // Declares new type Vector

let v = Vector(2.0, -5.0 * π);      // Structs created by initializing with type

// Only way to refer to this type is via its name
function magnitude(v: Vector): float {
  return hypot(v.i, v.y);
}                                   // type of magnitude is (Vector)->float
```

In Carlos, types are not first-class values. They can be named by identifiers, and you will see both type expressions and type names appear in the source code, but only in parameter declarations, the return type portion of a function declaration, in empty array expressions, with the `no` operator, and in casts. But you cannot assign them to variables or pass them to functions or return them from functions.

```
// let x = int                      // ERROR, int is not a legal expression value
// print(string)                    // ERROR, string is not a legal expression va
```

```
let supervisor = no string          // OK, empty optional of type string?
let scores = [float]()              // OK, just no scores yet
```

2.3 Declarations

A **declaration** binds an identifier to an entity. There are six kinds of declarations:

- type declarations
- function declarations
- variable declarations
- parameter declarations
- iterator declarations
- field declarations

Here is a short example showing all six kinds:

```
let friends = ["EY", "MA", "AA"];   // variable decl: friends
struct Vector {                     // type decl: Vector
  i: float                          // field decl: i
  j: float
}
for friend in friends {             // iterator decl: friend
  print(friend);
}
function triple(x: int): int {      // function decl: triple, parameter decl: x
  return x * 3;
}
```

Each occurrence of an identifier is either a **defining** occurrence or a **using** occurrence. Using occurrences are legal only in the **visible region** of the declaration that declares the identifier. In Carlos, **shadowing** is never permitted, so the visible region of a declaration coincides exactly with the declaration's **scope**. (This is different from most other static languages in which the visible region is the scope minus any "inner" scopes of declarations of identifiers with the same name and may therefore be discontinuous).

- The scope of an identifier declared in a variable, type, or function declaration begins with the declaration itself and extends to the end of the innermost block containing the declaration, or to the end of the program if the declaration does not appear in a block.

- The scope of an identifier declared in a field declaration is the same as the type in which the declaration appears.
- The scope of an identifier declared in a parameter declaration is the body of the innermost function in which the parameter declaration appears.
- The scope of an identifier declared as an iterator is the body of the loop statement in which it appears.

Example. In the following script:

```
function g(a: [int]) {}                          // line 1
let c = g([1, 2, 5]);                            // line 2
let y = c + 2;                                   // line 3
print(c);                                        // line 4
function f(string s) {                           // line 5
  // Note: let y = 0 would be illegal here       // line 6
  struct Point { x: int y: int }                 // line 7
  function q(): Point? { return no Point }       // line 8
  c = 50;                                        // line 9
}                                                // line 10
f("Can't see point or s here")                   // line 11
```

we have the following scopes:

Identifier	declared on line	has scope
g	1	1-11
a	1	body of g
c	2	2-11
y	3	3-11
f	5	5-11
s	5	5-10
Point	7	7-10
q	8	8-10

All declarations in a scope must declare distinct identifiers. Also notice that no shadowing rule only applies to a single path of nested scopes from the top-level scope. Identifiers in “sibling scopes” do

not shadow each other. In addition, struct fields are not part of any path, so there are no restrictions on their names:

```
function test(x: int, y: boolean) {
  const z = 0;
  // let x = "hello";          // ERROR: x is already a parameter
  if z > 1.0 {
    // let x = 2;              // ERROR: shadowing not allowed
    const a = 5;               // OK: new var, local to the if-block
  }
  struct Point {
    x: float                   // OK: fields not part of scope chain
    y: float                   // ...so they don't actually "shadow"
  }
  for i in 0 ..< 10 {
    // let i = 3;              // ERROR: in same scope as iterator i
    while false {
      // let i = 1;           // ERROR: no shadowing
      let a = false;           // OK: does not conflict with the other a
    }
  }                            // No var a is in scope out here though
}
```

Each variable, parameter, or field is constrained to hold values from a certain type. The type constraint for a variable is given in its initializing expression:

```
let gameOver = false;          // gameOver now constrained to booleans
gameOver = true;               // OK: true is boolean
// gameOver = 5.5;             // ERROR: gameOver can only take booleans.
```

Parameters and return types do not have initializers, so Carlos requires their types must be specified explicitly (while in principle, some complex type inference could be done, Carlos wants to be a smaller language that is easier to implement and restricts its type inference to very local situations):

```
function repeated(s: string; n: int): string {
  let result = "";
  repeat n {
    result = result + s;
```

```
  }
  return result;
}
```

2.4 Functions

As a simple language, Carlos does not have any anonymous functions; all functions must be named. The identifier used in the function declaration is essentially a read-only variable: you can assign it to other variables but you cannot reassign it:

```
function next(n: int): int {
  return n + 1;
}

let successor = next;                // OK
print(successor(2));                 // prints 3

// let next = successor;             // ERROR: next is read only
```

Functions are first-class objects, too. They can be assigned and passed as parameters and returned from functions:

```
function triple (x: int): int { return x * 3 }

function twice(f: (int)->int, x: int) { return f(f(x)) }

print(twice(triple, 8));             // prints "72"
```

2.5 Structs

It is possible to write any array, optional, or function type as a type expression from its constituent types; however there is no expression for a structure type. Structure types are defined in a `struct` declaration and bound to an identifier. The identifier is the only way to refer the type.

```
// We can use type expressions for function and array types
function applyToSecondInt(f: (int)->int, a: [int]): int {
  return f(a[1]);
}

struct Vector {i: float j: float}

// But for struct types we have to use a type name!
function magnitude(v: Vector): float {
  return hypot(v.i, v.y);
}
```

The fields within a struct type can be any identifier must be mutually distinct:

```
// struct S {x: int x: int}         // ERROR: duplicate field x
```

No field of a struct is allowed to have the type of the struct itself, as that would lead to an infinitely-sized structure. However, a struct may contain optionals, arrays, or functions that use the type itself:

```
struct S {
  a: S?           // OK: might be empty
  b: [S]          // OK: might be empty
  c: (S) -> S     // OK: function objects are not stored inline
  // d: S         // ERROR: struct would be infinite in size
}
```

2.7 Variables

A variable is something that stores a value. All variables are type-constrained. Variables are either writable (if declared with `let`) or not writable (if declared with `const`).

```
let turnsRemaining = 2;
const adjustment = 0.00137;

turnsRemaining = 3;                 // OK to assign, variable is writable
```

```
turnsRemaining--;                    // OK, ++ and -- are updates
// adjustment = 2.2E-7;              // ERROR: variable is read-only (const)
```

Variables directly store values of booleans, numbers, chars, and strings; however, they can only contain *references* to arrays, structs, functions, and optionals. This means that aliasing of an object can occur:

```
let a = [1, 2, 3]
let b = a;                           // same object pointed to by a and b
a[2] = 1000;                         // updates the object
print(b);                            // prints [1, 2, 1000]
```

The kinds of variables are:

- i

  Simple variable. Identifier i must be bound to a visible variable, parameter, iterator, or function declaration. (This means stand-alone fields are *not* variables, nor are identifiers associated with types.) The variable's type is the type from the declaration, and it is writable iff its declaration used `let`.

- v `.` f

  Member variable. v must be a variable of a struct type. f must be an identifier declared as a field of v's type. The resulting variable refers to the f-field of the object referred to by v, is writable iff v is writable, and its type is the type associated with the field f.

- v `[` e `]`

  Subscripted variable. v must be a variable of an array type, and e must have type `int`. The result is the array component at (zero-based) index e, is writable iff v is writable, and its type is the base type of the array type. If during execution e evaluates to a value less than zero or greater than or equal to the length of v, the program dies.

- v `(` $e_1 \ldots e_n$ `)`

  Call expression. v must either (1) have type $(T_1, \ldots T_n) \rightarrow T_0$ where each e_i has type T_i, or (2) be an identifier bound to a structure type. In the former case, the arguments are evaluated in any order and the function is called with the arguments copied to the parameters, with the result being the return value of the function and its type is the return type of the function type. In the latter case the variable is a newly created struct instance of the given type; the arguments are evaluated and assigned to the fields of the new object. In both cases, the resulting variable is **not** writable.

- *v*`?.`*f*

  Optional Member variable. *v* must be a variable of a optional type wrapping a struct type. *f* must be an identifier declared as a field of *v*'s wrapped type. The resulting variable refers to the *f*-field of the object referred to by *v* if something is wrapped or an empty optional otherwise, is writable iff *v* is writable, and its type is the optional type wrapping the type associated with the field *f*.



```
struct Point {
  x: float
  y: float
}

let p = some Point(3, 5)
let q = no Point

print(p?.x)             // type is int?, value is (some 5)
print(q?.x)             // type is int?, value is (no int)
```

- *v*`?[`*e*`]`

  Optional subscript variable. *v* must be a variable of a optional type wrapping an array type. *e* must have type `int`. The result is the element at (zero-based) index *e* if an array is wrapped or an empty optional otherwise, is writable iff *v* is writable, and its type is the base type of the wrapped array's type. If during execution *e* evaluates to a value less than zero or greater than or equal to the length of *v*, the program dies.



```
let p = some [3, 5, 8]
let q = no [int]

print(p?[1])            // type is int?, value is (some 5)
print(q?[1])            // type is int?, value is (no int)
```

Variables are given values (1) in a variable declaration, (2) in an assignment statement, or (3) if a parameter, during a function call where arguments are assigned to parameters. In all cases, the type *S* of the value being assigned (the source) is checked to see if it is assignable to the type constraint *T* on the variable (the target). The assignment is valid if and only if *S* is **assignable** to *T*, which is the case if and only if:

- *S* is *T*.
- *T* is `any`.

- S is $(S_1, \ldots, S_n) \to S_0$ and T is $(T_1, \ldots, T_n) \to T_0$ and S_0 is assignable to T_0 and each T_i is assignable to S_i.

That is, Carlos covariant in return types, contravariant in parameters, and invariant elsewhere.

2.7 Statements

A statement is code that is executed solely for its side effect; it produces no value. The kinds of statements are:

- `let` i `=` e `;`

  `const` i `=` e `;`

  Variable declaration statement. Declares a new variable with name i. This declaration must be unique within its scope. Evaluates e and assigns this value to the new variable. The new variable will be constrained to be assigned only variables of e's type. The variable is writable iff declared with `let`.

- `struct` S `{` $x_1 : T_1 \ldots x_n : T_n$ `}`

  Struct type declaration. Defines a new struct type named S with the given fields. The identifier S must be unique within its scope, and the fields $x_1 \ldots x_n$ must be mutually distinct. None of the T_i's are allowed to be S itself.

- `function` f `(` $x_1 : T_1 \ldots x_n : T_n$ `)` `:` T_0 b

  Function declaration. Defines a new function named f with type $(T_1, \ldots, T_n) \to T_0$. The identifier f must be unique within its scope. The parameters must be unique within their scope.

- v `=` e `;`

  Assignment statement. e must have the same type that v is constrained to. v is determined and then e is evaluated, then the value of e is copied into v.

- v`++;`

  Increment statement. v must have type `int`. Increments v.

- v`--;`

  Decrement statement. v must have type `int`. Decrements v.

- f`(` $e_1, \ldots, e_n$ `);`

Call statement. f must have type $(T_1, \ldots T_n) \rightarrow$ `void` where each e_i has type T_i. The arguments are evaluated in any order and the function f is called with the arguments copied to the parameters.

- `break;`

  Break statement. This statement may only appear within a loop (`for` , `while` , or `repeat`) statement that is properly within the same function as the break statement. The break statement terminates the execution of the innermost enclosing loop.

- `return;`

  Short Return statement. Causes an immediate return from the innermost enclosing function, whose type must have a return type of `void` .

- `return` e `;`

  Return statement. Evaluates e then causes the innermost enclosing function to immediately return the value of e. e must have the type of the function's type's return type.

- `if` e b

  Short If statement. e must have type `boolean` . If e evaluates to true, b is executed, otherwise nothing else happens.

- `if` e b `else` s

  If statement. e must have type `boolean` . If e evaluates to true, b is executed, otherwise s is executed.

- `while` e b

  While statement. e must have type `boolean` . First e is evaluated. If e produces false the execution of the while statement terminates. If e produces true, b is executed then the entire while-statement is executed again.

- `repeat` e b

  Repeat statement. e is evaluated and must have type `int` . Body b is executed the number of times equal to the value of e. If this value is negative the loop executes zero times.

- `for` i `in` r b

  For-each statement. r must have an array type or string type. r is evaluated and then b is executed repeatedly with i bound to successive elements from r if r is an array, and for successive code points from r if r is a string.

- `for` i `in` e_1 `...` e_2 b

  Through-range iteration statement. e_1 and e_2 must have type `int`. They are first evaluated in any order and then b is executed repeatedly with i bound to successive elements in the range, which goes from e_1 to e_2 inclusive.

- `for` i `in` e_1 `..<` e_2 b

  Upto-range iteration statement. e_1 and e_2 must have type `int`. They are first evaluated in any order and then b is executed repeatedly with i bound to successive elements in the range, which goes from e_1 to e_2 exclusive.

2.8 Expressions

An expression evaluates to a value or crashes the program. Carlos is a statically-typed language, meaning that the type of every expression can be inferred by looking at the program text; there is no need to run the program to determining an expression's type.

The Carlos expressions are:

- An integer literal, which has type `int`.
- A floating point literal, which has type `float`.
- A string literal, which has type `string`.

  String literals are made from a sequence of character specifications enclosed in quotation marks (U+0022). A character specification is either a non-control character, an extended grapheme cluster, or an escape sequence. An escape sequence is one of:

Escape	**Description**
`\n`	newline (U+000A)
`\t`	tab (U+0009)
`\u{` *xxxxxx* `}`	where *xxxxxx* is a one to six character hexadecimal digit sequence standing for a character with a given code point.
`\"`	the quotation mark character (U+0022)
`\'`	the apostrophe character (U+0027)
`\\`	the backslash character (U+005C)

- `true`

  The literal of type `boolean` denoting truth.

- `false`

  The literal of type `boolean` denoting falsity.

- v

  v is a variable. The type of this expression is the type of the variable v, and the value of this expression is the current value stored in v.

- `[` $e_1, \ldots, e_n$ `]`

  Array object construction. Each e_i must have exactly the same type T (there is no allowance for subtyping or anything like that). This expression refers to a newly constructed array of n items consisting of the values of each subexpression, respectively, and has type $[T]$.

- `[` T `]()`

  Empty array. A newly constructed empty array of type $[T]$.

- `some` e

  An optional that wraps e.

- `no` T

  The empty optional of type $T?$.

- `#` e

  Here e must be an expression of some array type. Produces the length of the array. The type of this expression is `int`.

- `random` e

  Here e must be an expression of some array type. Produces a random element from the array.

- `(` e `)`

  Evaluates e and produces its value.

- `-` e

  e must have type `int` or `float`. Evaluates e and produces the negation of e.

- `~` e

e must have type `int`. Evaluates *e* and produces the bitwise complement of *e*.

- `! e`

  e must have type boolean. If *e* evaluates to true, the entire expression produces **false**, otherwise it produces **true**.

- e_1 `*` e_2

  Both subexpressions must have type `int` or both must have type `float`. The subexpressions are evaluated in any order and their product is produced.

- e_1 `/` e_2

  Both subexpressions must have type `int` or both must have type `float`. The subexpressions are evaluated in any order and their quotient is produced. The quotient will have the same type as the operands, so division of integers produces the floor division.

- e_1 `%` e_2

  Each subexpression must have type `int`. Both expressions are evaluated, in any order, and the entire expression produces an integer which is the remainder of e_1 divided by e_2.

- e_1 `+` e_2

  Both subexpressions must have type `int` or both must have type `float` or both must have type `string`. The subexpressions are evaluated in any order and their sum (for numbers) or concatenation (for strings) is produced.

- e_1 `-` e_2

  Both subexpressions must have type `int` or both must have type `float`. The subexpressions are evaluated in any order and their difference is produced.

- e_1 `<<` e_2

  Each e_i must have type `int`. Produces the value of e_1 shifted left e_2 positions.

- e_1 `>>` e_2

  Each e_i must have type `int`. Produces the value of e_1 arithmetically shifted right e_2 positions.

- e_1 `<=` e_2

Each subexpression must have type `int`, or must both have type `float`, or must both have type `string`. Both expressions are evaluated, in any order, and the entire expression produces whether the value of e_1 is less than or equal to the value of e_2.

- e_1 `<` e_2

  Each subexpression must have type `int`, or must both have type `float`, or must both have type `string`. Both expressions are evaluated, in any order, and the entire expression produces whether the value of e_1 is less than the value of e_2.

- e_1 `==` e_2

  e_1 and e_2 must have the same type. The subexpressions are evaluated in any order, and the entire expression produces whether these values are equal, defined as follows. Ints, floats, and booleans are equal if they have the same value. Strings are the same if their byte sequences are equal (they are *not* normalized prior to comparison. Arrays, structs, and functions are equal iff the two expressions reference the exact same object. Two optionals are equal iff they are both empty optionals or their wrapped values are equal.

- e_1 `!=` e_2

  Equivalent to `!(e1==e2)`.

- e_1 `>` e_2

  Each subexpression must have type `int`, or must both have type `float`, or must both have type `string`. Both expressions are evaluated, in any order, and the entire expression produces whether the value of e_1 is greater than the value of e_2.

- e_1 `>=` e_2

  Each subexpression must have type `int`, or must both have type `float`, or must both have type `string`. Both expressions are evaluated, in any order, and the entire expression produces whether the value of e_1 is greater than or equal to the value of e_2.

- e_1 `&` e_2

  Each subexpression must have type `int`. Both expressions are evaluated, in any order, and the entire expression produces an int which is the bitwise AND of e_1 and e_2.

- e_1 `^` e_2

  Each subexpression must have type `int`. Both expressions are evaluated, in any order, and the entire expression produces an int which is the bitwise XOR of e_1 and e_2.

- e_1 `|` e_2

Each subexpression must have type `int`. Both expressions are evaluated, in any order, and the entire expression produces an int which is the bitwise (inclusive) OR of e_1 and e_2.

- e_1 `&&` e_2

  Each subexpression must have type `boolean`. First e_1 is evaluated. If it evaluates to false, the entire expression immediately produces false (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

- e_1 `||` e_2

  Each subexpression must have type `boolean`. First e_1 is evaluated. If it evaluates to true, the entire expression immediately produces true (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

- e_1 `??` e_2

  e_1 must have type T`?` for some type T and e_2 must have type T. First e_1 is evaluated. If it wraps a value, the entire expression immediately produces the wrapped value (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

3 Syntax

The source of a Carlos program is a Unicode string. Here is the grammar in Ohm notation:

carlos.ohm

```
Carlos {
  Program     = Statement+

  Statement   = VarDecl
              | TypeDecl
              | FunDecl
              | Exp9 ("++" | "--") ";"                    --bump
              | Exp9 "=" Exp ";"                          --assign
              | Exp9_call ";"                             --call
              | break ";"                                 --break
              | return Exp ";"                            --return
              | return ";"                                --shortreturn
```

```
            | IfStmt
            | LoopStmt

VarDecl     = (let | const) id "=" Exp ";"
TypeDecl    = struct id "{" Field* "}"
Field       = id ":" Type
FunDecl     = function id Params (":" Type)? Block
Params      = "(" ListOf<Param, ","> ")"
Param       = id ":" Type

Type        = Type "?"                                  --optional
            | "[" Type "]"                              --array
            | "(" ListOf<Type, ","> ")" "->" Type       --function
            | id                                        --id

IfStmt      = if Exp Block else Block                   --long
            | if Exp Block else IfStmt                  --elsif
            | if Exp Block                              --short
LoopStmt    = while Exp Block                           --while
            | repeat Exp Block                          --repeat
            | for id in Exp ("..." | "..<") Exp Block   --range
            | for id in Exp Block                       --collection
Block       = "{" Statement* "}"

Exp         = Exp1 "?" Exp1 ":" Exp                     --conditional
            | Exp1
Exp1        = Exp1 "??" Exp2                            --unwrapelse
            | Exp2
Exp2        = Exp3 ("||" Exp3)+                         --or
            | Exp3 ("&&" Exp3)+                         --and
            | Exp3
Exp3        = Exp4 ("|" Exp4)+                          --bitor
            | Exp4 ("^" Exp4)+                          --bitxor
            | Exp4 ("&" Exp4)+                          --bitand
            | Exp4
Exp4        = Exp5 ("<="|"<"|"=="|"!="|">="|">") Exp5   --compare
            | Exp5
Exp5        = Exp5 ("<<" | ">>") Exp6                   --shift
            | Exp6
```

```
Exp6        = Exp6 ("+" | "-") Exp7                          --add
            | Exp7
Exp7        = Exp7 ("*"| "/" | "%") Exp8                     --multiply
            | Exp8
Exp8        = Exp9 "**" Exp8                                 --power
            | Exp9
            | ("#" | "-" | "!" | some | random) Exp9         --unary
Exp9        = true ~mut
            | false ~mut
            | floatlit ~mut
            | intlit ~mut
            | no Type ~mut                                   --emptyopt
            | Exp9 ("(" | "?(") ListOf<Exp, ","> ")" ~mut    --call
            | Exp9 ("[" | "?[") Exp "]"                      --subscript
            | Exp9 ("." | "?.") id                           --member
            | stringlit ~mut
            | id                                             --id
            | Type_array "(" ")" ~mut                        --emptyarray
            | "[" NonemptyListOf<Exp, ","> "]" ~mut          --arrayexp
            | "(" Exp ")" ~mut                               --parens

intlit      = digit+
floatlit    = digit+ "." digit+ (("E" | "e") ("+" | "-")? digit+)?
stringlit   = "\"" char* "\""
char        = ~control ~"\\" ~"\"" any
            | "\\" ("n" | "t" | "\"" | "\\")                 --escape
            | "\\u{" hex hex? hex? hex? hex? hex? "}"         --codepoint
control     = "\x00".."\x1f" | "\x80".."\x9f"
hex         = hexDigit
mut         = ~"==" "=" | "++" | "--"

let         = "let" ~alnum
const       = "const" ~alnum
struct      = "struct" ~alnum
function    = "function" ~alnum
if          = "if" ~alnum
else        = "else" ~alnum
while       = "while" ~alnum
repeat      = "repeat" ~alnum
```

```
  for         = "for" ~alnum
  in          = "in" ~alnum
  random      = "random" ~alnum
  break       = "break" ~alnum
  return      = "return" ~alnum
  some        = "some" ~alnum
  no          = "no" ~alnum
  true        = "true" ~alnum
  false       = "false" ~alnum
  keyword     = let | const | struct | function | if | else | while | repeat
              | for | in | break | return | some | no | random | true | false
  id          = ~keyword letter alnum*

  space      += "//" (~"\n" any)*                              --comment
}
```

4 Standard Library

The following identifiers are pre-defined in a scope that surrounds the program. This means that none of these identifiers may be declared anywhere in a program (but they may be used as fields, though doing so is not recommended).

- `function print(s: any): void`

  Writes a representation of s to standard output.

- `function codepoints(s: string): [int]`

  Returns the code points of the s.

- `function bytes(s: string): [int]`

  Returns the bytes of the UTF-8 encoding of s.

- `const π`

  Returns the best approximate value of π in the type `float`.

- `function sqrt(x: float): float`

Returns the square root of x.

- `function sin(x: float): float`

  Returns the sine of x radians.

- `function cos(x: float): float`

  Returns the cosine of x radians.

- `function exp(x: float): float`

  Returns e^x.

- `function ln(x: float): float`

  Returns the natural log of x.

- `function hypot(x: float, y: float): float`

  Returns the hypotenuse of a right triangle with sides $|x|$ and $|y|$.

Syntax

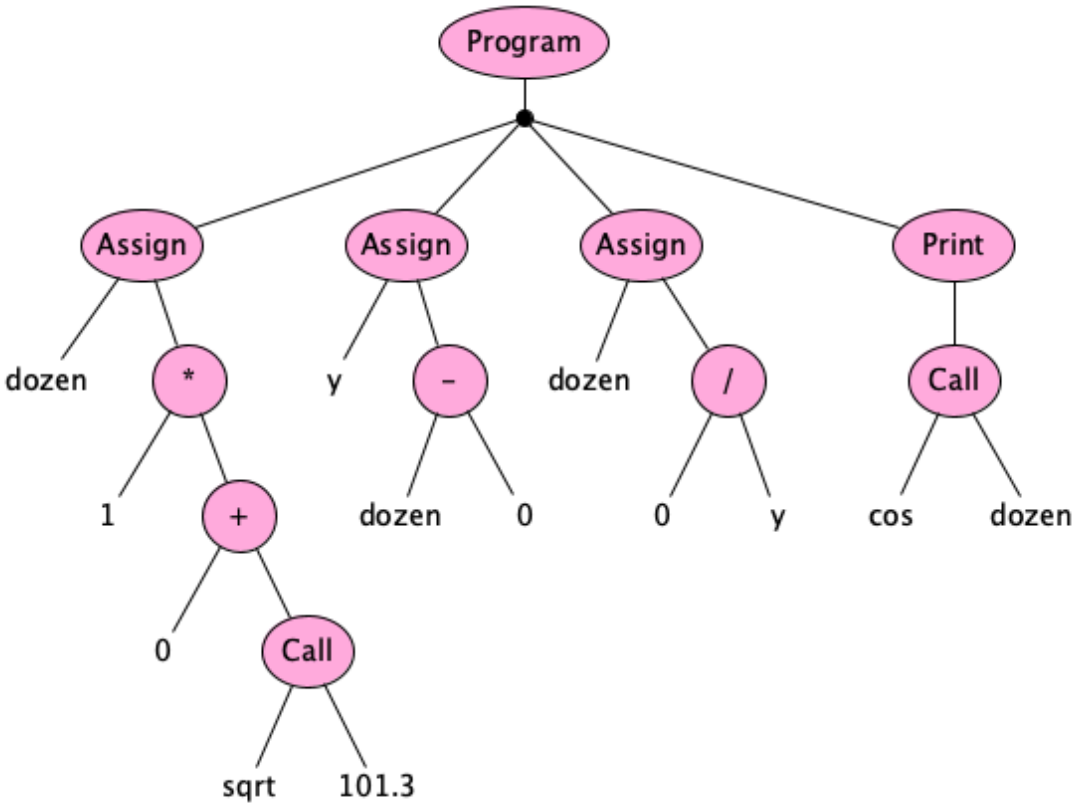
*What is **syntax** and how does it differ, from say, semantics? What are the different ways that we can define a syntax? What is the difference between concrete and abstract syntax? And why do programming languages tend to use context-free grammars for their syntax?*

CONTENTS

Motivation • Definition • Learning by Doing • Lexical and Phrase Syntax • Dealing With Ambiguity • Grammars for Programming Languages • The Problem of Context • Abstract Syntax • Syntax in the Real World • Alternate Syntactic Descriptions • Recall Practice • Summary

Motivation

A language gives us a way structure our thoughts. Utterances in a language (called **programs** in a programming language) have a kind of internal structure, for example:



We generally capture this structure as a sequence of symbols; for example, the structure above might appear as:

```
dozen = 1 * (0 + sqrt(101.3));
y = dozen - 0;    // TADA 🥑
dozen = 0 / y;
print cos(dozen);//
```

or even:

```
MOVE PRODUCT OF 1 AND (SUM OF 0 AND SQRT OF 101.3) INTO DOZEN.
MOVE DIFFERENCE OF DOZEN AND 0 INTO Y.
COMMENT TADA 🥑
MOVE QUOTIENT OF 0 AND Y INTO DOZEN.
PRINT COSINE OF DOZEN.
COMMENT
```

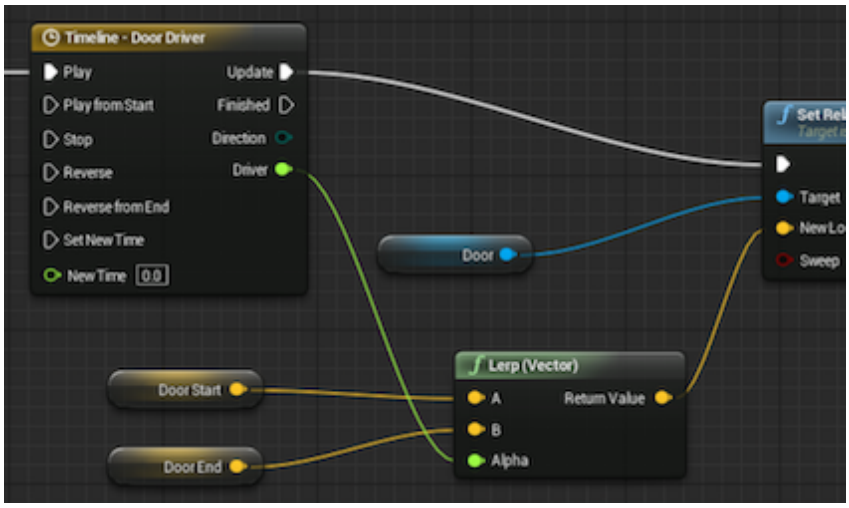
or even:

```
(program
  (assign dozen (* 1 (+ 0 (call sqrt 101.3))))
```

```
(assign y (- dozen 0))          ; TADA 🥑
(assign dozen (/ 0 y))
(print (call cos dozen))        ;
```

Of course, we don't have to use strings! Visual languages do exist:





> ***Text vs. Pictures***
>
> *Text is more dense than pictures, and more directly machine readable at times (no computer vision machine learning necessary). Even if your language was expressed with pictures, there will be an underlying byte-array, or string, representation for storage and transmission.*
>
> *So our focus from now on will be on text.*

***Exercise**: Text is by no means always better than pictures! Please watch Bret Victor's [The Future of Programming] for thoughts on programming visually.*

If you were a language designer, you'd deal with questions such as:

- Is code expressed in terms of blocks? Visual graphs? Text?
- If text, which characters are allowed? Is whitespace significant? Is case significant?
- Is there a lot of punctuation? Symbols? Or is it mostly words? If words, what natural language are you biased toward?
- How do we do comments? Pragmas? Directives?
- Do we express structure with indentation, curly-braces, terminating-end, or nested parentheses?
- Should operations have a functional (`length(a)`) or message-oriented (`a.length`) feel?
- What kind of shorthands and idioms should be made available?
- How do program units map to files or other physical containers?

Definition

The **syntax** of a programming language is the set of rules that define which arrangements of symbols comprise structurally legal programs—and for each structurally legal program, what its structure actually is.

Syntactic rules should be able to tell us that these sentences are not well-formed in English:

- `telescope a they the With hit saw.`
- `yqvfe.,--)AO7%;;;;;;;; ; !dogs`

but that these are:

- `They hit the telescope with a saw.`
- `They saw the hit with a telescope.`

and so is this famous one, due to Chomsky:

- `Colorless green ideas sleep furiously.`

Learning by Doing

Let's learn about syntax by going through the process of designing a syntax for a small language.

We'll assume the preliminary phases of language design—identifying the purpose (education), scope (simple arithmetic computation), audience (students), and name (Astro) of our language—have been done, and now it is time for the technical work to begin.

We first sketch a program to illustrate the syntactic features we want:

```
// A simple program in Astro

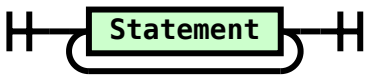
radius = 55.2 * (-cos(2.8E-20) + 89) % 21;    // assignment statement
the_area = π * radius ** 2;                   // a built-in identifier
print hypot(2.28, 3 - radius) / the_area;     // print statement
```

Next, we put our ideas into words. A first pass: "Programs are structured as a sequence of one or more statements, each of which is an assignment or print statement, with expressions formed with numbers, variables, function calls, and the usual arithmetic operators, which can have parentheses when needed. Comments look like the slash-slash comments of C-like languages."

Natural language isn't super precise, so as we sometimes do in design, we'll next turn thoughts into pictures.

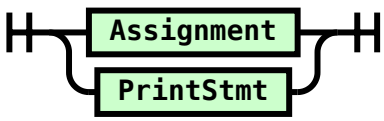
A program is a sequence of one or more statements:

Program



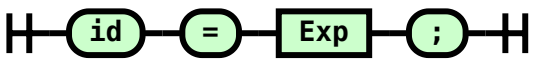
There will be two kinds of statements: assignments and print statements:

Statement

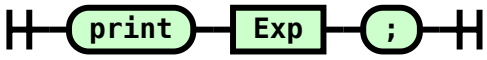


Assignments and print statements look like this:

Assignment



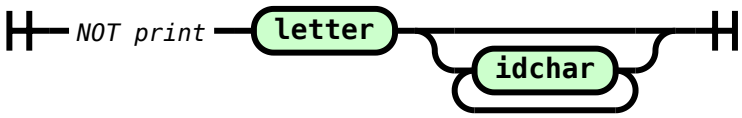
PrintStmt



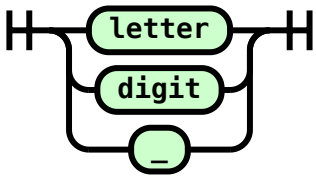
An **identifier** is the computer science word for a *name* you attach to an **entity** (a variable, constant, function, type, parameter, or similar thing). A **keyword** is a special word in your language, like `let` or `if` or `while` —these words generally do not name things—they are just used to structure the code. If a keyword is not allowed to be an identifier, we call it a **reserved word**. Astro has one reserved word: `print`.

Identifiers in Astro begin with a letter, and can have letters, digits, and underscores (examples: `x`, `last_attempt`, `p1`, `p2`, `overTheLimit`, `bot8675309_jEnNy`). We will call letters, digits, and underscores **identifier characters** (`idchar`s). But notice that `print` is not allowed to be an identifier:

id



idchar



How is the reserved word `print` specified? We have to be careful. It's not just the five letters p, r, i, n, and t. If it were then the program:

```
printy;
```

would be legal! It would be the five characters spelling print followed by a legal expression, namely the identifer ***y***. We want the word `print` to not bleed into any following characters that might be part of an expression. In other words, `print` must not be immediately followed by an identifier character:

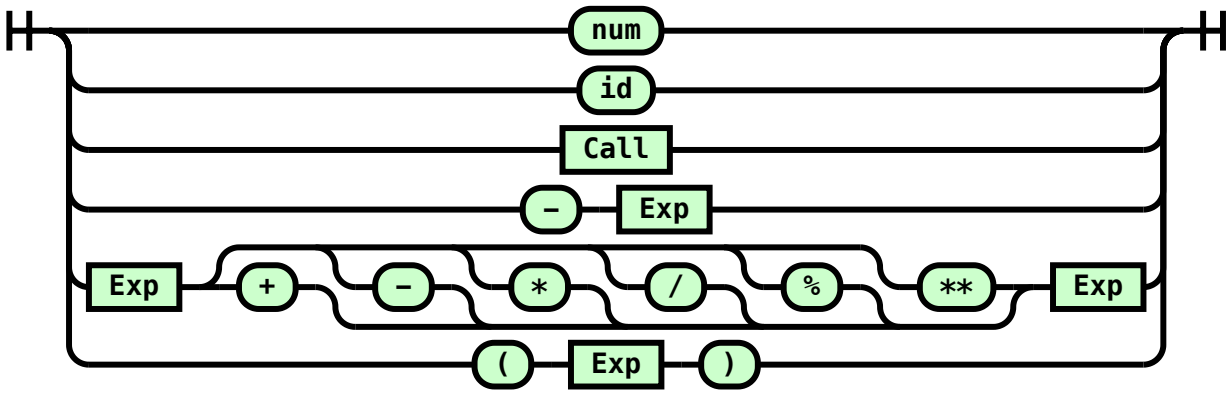
print



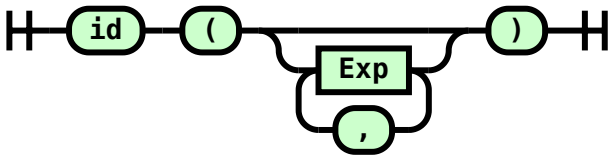
***Exercise**: Make sure you understand how this definition ensures that "printy" is an identifier and not* `print` *followed by the identifier* `y`.

A **statement** performs an action (perhaps with a side effect), but **expressions** represent the computation of a value. Expressions can be numbers, identifiers (representing variables), function calls, or can be made up of operators applied to other expressions. We can put parentheses around any expression. This will do the trick (but spoiler alert: we will have to change it later):

`Exp`



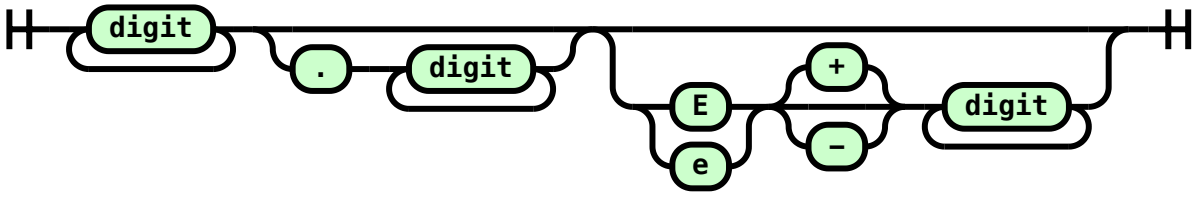
`Call`



***Exercise**: Explain the difference between expressions and statements to a friend. Make yourself a sacred vow never to mix the terms up. Precise language is a good thing. Sometimes we slip up, but still, let's try not to say things like "plus statement."*

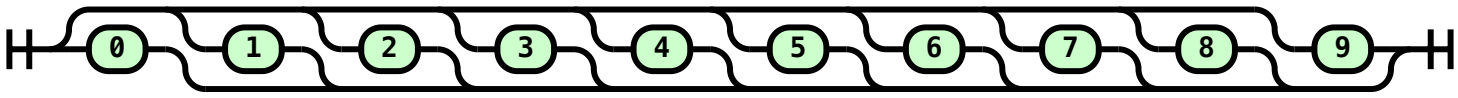
We mentioned numerals here, so let's define these. We'll keep things in decimal only (no worries about hex or binary), and use the times-ten-to-the notation from popular programming languages:

`num`



What about letters and digits? Well digits are pretty easy:

digit

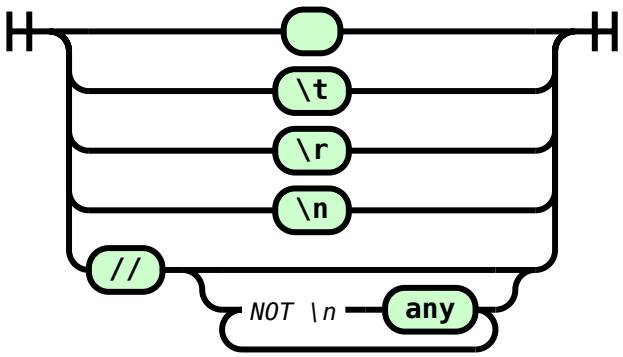


For letters, well, there are tens of thousands of these in Unicode, so we'll just say the rule for `letter` is "built-in."

Lexical and Phrase Syntax

Did you notice that we used two kinds of nodes in our syntax diagrams? We used ovals for constructs that internally cannot have spaces in them; we call these **lexical nodes**, or **tokens**. We used rectangles for constructs whose components can have spaces between them; we call these **phrase nodes**, or **phrases**. The special category called `space` defines what can separate tokens within phrases.

space



In the little language Astro that we are designing, the separators are the space character, the tab character, the carriage return character, the linefeed character (the linefeed (U+000A) is pretty much universally accepted as the "newline"), and comments. Comments function as spaces, too! For comments, we chose to start with `//` and go to the end of the current line.

We also followed an accepted convention to start the names of lexical nodes with lowercase letters and capitalize the names of phrase nodes.

Another convention we will follow for programming language syntax is to take the following as predefined:

- `letter`, for any Unicode letter

- `digit`, for `"0".."9"`
- `alnum`, for `letter | digit`
- `upper`, for any Unicode uppercase letter
- `lower`, for any Unicode lowercase letter
- `hexDigit`, for `digit | "a".."f" | "A".."F"`
- `any`, for any Unicode character at all

and also allow these conveniences:

- `\u{` and `}`, delimit a code point in hex and stands for the unicode character with that code point
- `\n`, for the LF character (same as `\u{a}`)
- `\r`, for the CR character (same as `\u{d}`)
- `\t`, for the tab character (same as `\u{9}`)
- `\"`, for the quote character (same as `\u{22}`)
- `\'`, for the apostrophe character (same as `\u{27}`)
- `\\`, for the backslash character itself (same as `\u{5c}`)

Note that `any` can be defined as `\u{0}..\u{10ffff}`.

> ***Remember***
>
> *The term* `space`*, though it looks like a lexical node, is something meta; it is treated specially.*

Let's spend some more time on the philosophical significance of the separation of lexical and phrase syntax.

The lexical syntax, with the exception of the special rule for `space`, show us how to combine characters into words and punctuation which we'll call **tokens**. The phrase syntax show us how to combine words and symbols into **phrases** (where the words and symbols can, if you wish, be surrounded by spacing).

> ***Exercise**: Meditate on how profound this is. Have you seen anything like this before? Hint: Are you reading this page in English?*

Now if we wanted to **recognize** whether a string is "in" a language, we can figure this out in two phases. The first phases uses the lexical rules to **tokenize** a stream of characters into a stream of

tokens, and the second phase uses the phrase rules to **parse** the token stream into the **parse tree**. Here's an example. The source string:

```
  print( // 麻🌿
420 );
```

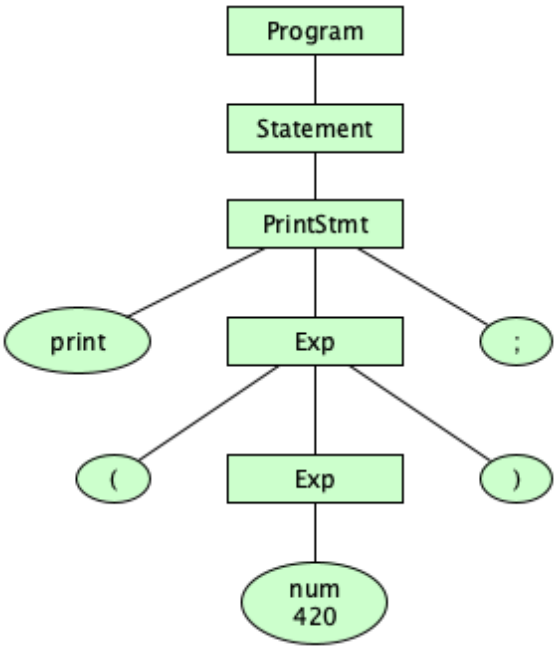
is made up of these characters:

SPACE *SPACE* *LATIN SMALL LETTER P* *LATIN SMALL LETTER R* *LATIN SMALL LETTER I* *LATIN SMALL LETTER N* *LATIN SMALL LETTER T* *LEFT PARENTHESIS* *TAB* *SOLIDUS* *SOLIDUS* *SPACE* *KANGXI RADICAL HEMP* *HERB* *LINEFEED* *DIGIT FOUR* *DIGIT TWO* *DIGIT ZERO* *SPACE* *RIGHT PARENTHESIS* *SEMICOLON*

When we follow the lexical rules to this character stream, skipping the spaces (and comments), we get the token stream:

`print` `(` `num(420)` `)` `;`

Next, we follow the phrase rules allows us to uncover the parse tree:



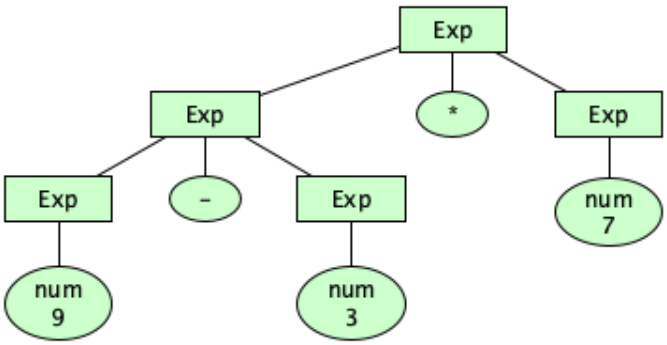
A very important thing to note: **The frontier of the parse tree is the token stream.**

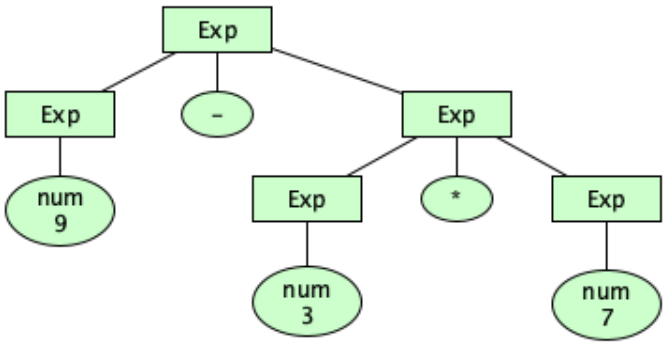
> ***Exercise**: Repeat this phrase to yourself five times: The frontier of the parse tree is the token stream.*

> ***The parse tree ends at tokens, not characters***
>
> *Please take the time to consider how silly it would be if the parse tree expanded all of the lexical rules down to characters. Having the parse tree stop at tokens is a good example of what we would call "breaking a complex problem down into simpler parts."*

Dealing With Ambiguity

How are we doing so far? We are able to distinguish well-structured Astro programs from all other Unicode strings. But there are some things we haven't dealt with yet. For one, we have some strings with multiple structural forms. For example, the phrase `9-3*7` can be **parsed** in two ways:





Having more than one parse tree for a given input string means that our syntax description is **ambiguous**. It's possible to handle this particular kind of ambiguity in the syntax. Here's how.

Precedence

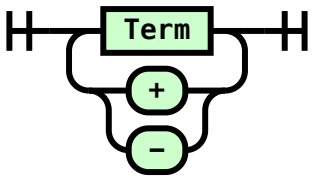
We can create rules that force certain operators to be applied before others; that is, the **precedence** of operators can be enforced in our syntax definition. To do so, we define additional syntactic categories. We say:

- An expression is a sequence of one or more **terms**, separated by additive operators.
- A term is a sequence of one or more **factors**, separated by multiplicative operators.
- A factor is made up of **primaries**, separated by exponentiation operators, or a factor can just be a negated primary.

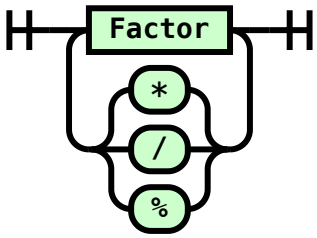
- A primary is the most basic expression possible: either a simple identfier, a numeral, a call, or a parenthesized expression. Parentheses allow unlimited nesting.

We can write:

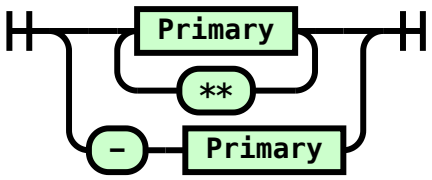
Exp



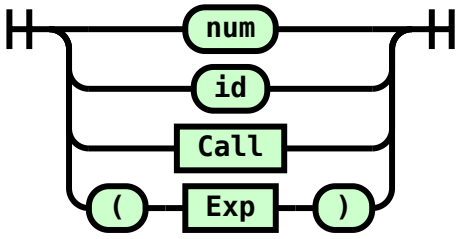
Term



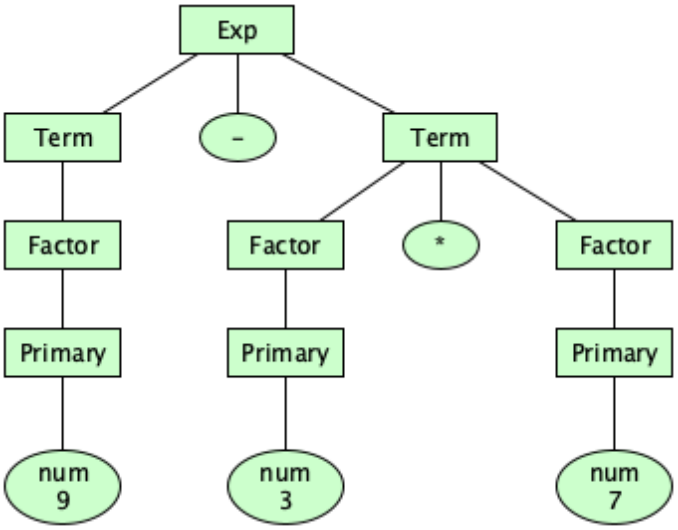
Factor



Primary



Great! Now there is one and only one parse tree for that previously problematic expression:



Note that the new syntax has forced the binary operators into a precedence hierarchy!

- `+` and `-` have the lowest precedence.
- `*`, `/`, and `/` have the next higher precedence.
- `**` has the highest precedence.

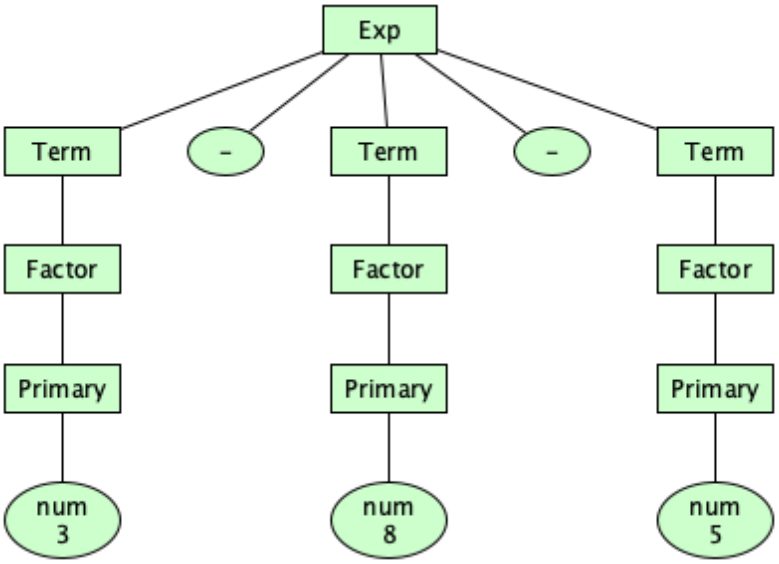
Of course, you can think of parenthesized expressions as being done before anything else, though we don't usually think of these as operators.

> ***Exercise**: Precedence is usually a concept that applies only to binary operators, but they way in which unary operators mix with binary operators does need to be addressed. For instance, how should `-3**2` be parsed? In Python exponentiation precedes negation, like `-(3**2)`. In Elm, negation precedes exponentiation, like `(-3)**2`. Astro follows JavaScript and simply does not allow this expression (forcing programmers to use parentheses in this case)! Show how this is done.*

> ***Exercise**: Our syntax diagram above does not allow `-3**2` as an expression, but it does allow `-3+2` and `-3*2`. Why did we care only enough to ensure negation did not mix with exponentiation, but we were fine with it mixing with addition and multiplication?*

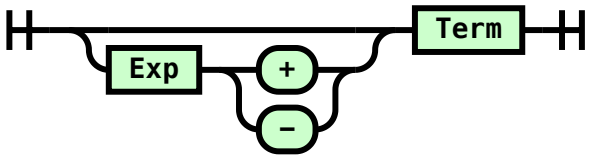
Associativity

Wait, we are not done with structuring our operators just yet. The way things stand now, the parse tree for `3-8-5` looks pretty flat:

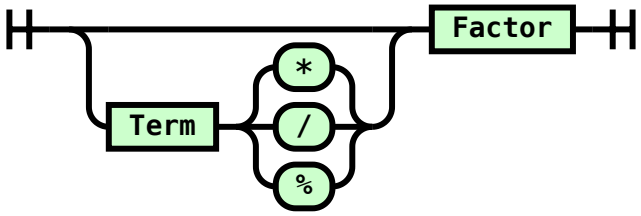


It doesn't suggest whether we mean to compute `(3-8)-5` (which would be -10) or `3-(8-5)` (which would be 0). We can give a syntax that makes this clear. In our design, let's make the additive and multiplicative operators **left-associative** and the exponentiation operator **right-associative**:

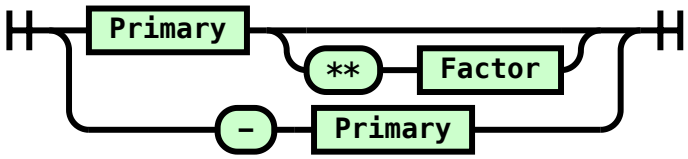
`Exp`



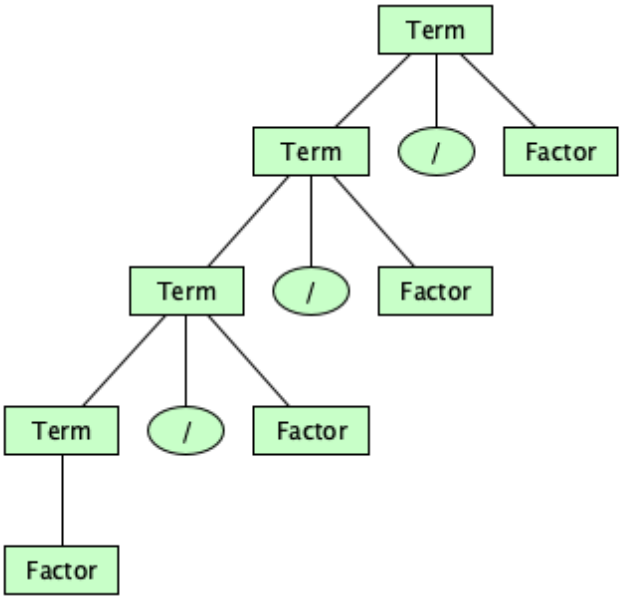
`Term`

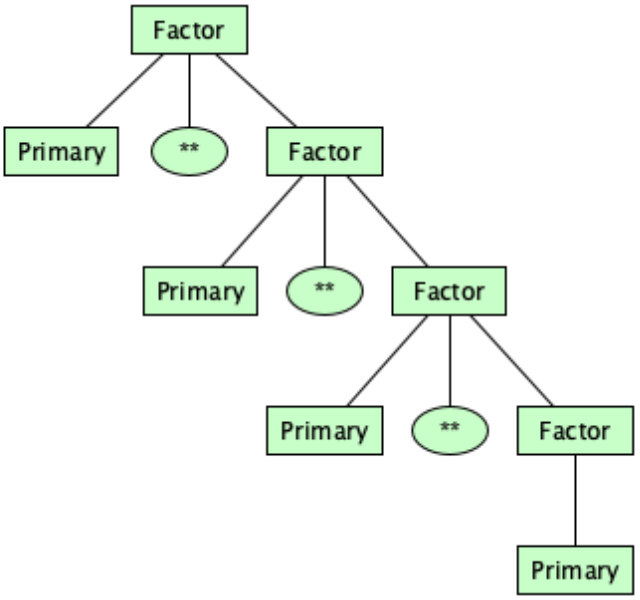


`Factor`



How the heck does this work? Study these parse trees, and hopefully the insight will come to you! (Hint: remember the syntax is *designed* to force the tree to "come out" a certain way.





***Exercise**: Study this until you understand it well.*

Grammars for Programming Languages

Syntax Diagrams are fine for giving an overview of program structure, but, like all pictorial notations, they can take up a lot of space and are rather hard to process by a machine. If we could write a denser, text-based description of a syntax, we could write a program that checked a candidate program for conformance with the syntax! This is exactly what compilers, interpreters, and related tools do.

Let's translate the syntax diagrams above into a grammar for Astro. We will use an analytic grammar, rather than a generative one, as we want to take advantage of negative lookahead to help describe our keywords and identifiers.

Program	⟵	*Statement*$^+$
Statement	⟵	*Assignment* / *PrintStmt*
Assignment	⟵	*id* "=" *Exp* ";"
PrintStmt	⟵	*print Exp* ";"
Exp	⟵	(*Exp* ("+"/"-"))? *Term*
Term	⟵	(*Term* ("*"/"/"/"%"))? *Factor*
Factor	⟵	*Primary* ("**" *Factor*)? / "-" *Primary*
Primary	⟵	*Call* / *id* / *num* / "(" *Exp* ")"
Call	⟵	*id* "(" (*Exp* ("," *Exp*)*)? ")"
numeral	⟵	*digit*$^+$ ("." *digit*$^+$)? (("E"/"e") ("+"/"−")? *digit*$^+$)?
id	⟵	∼*print letter idchar**
idchar	⟵	*letter* / *digit* / "_"
print	⟵	"print" ∼*idchar*
space	⟵	" " / "\t" / "\r" / "\n" / "//" (∼"\n" *any*)*

***Exercise**: Because we used an analytic grammar, it was imperative that we put the Call choice before the id choice in the Primary rule. What would happen if we reversed the order of these two choices?*

***Exercise**: Compare, in detail, the grammar for Astro with the syntax diagrams we've been using. Do you see how they are related?*

The Problem of Context

Question: Does the grammar we have defined above for Astro capture the following (desirable) rule:

> "You can only use an identifier if it has been previously assigned to."

Answer: It does not.

***Exercise**: Show that it does not. Hint: Is `print x;` a legal program according to the grammar?*

***Exercise**: Show how to capture the rule in a grammar. Hint: Do not be frustrated if you cannot.*

Enforcing this rule requires knowledge of **context**. That is, the syntactic rule for producing expressions such as calls and arithmetic operations would need to somehow know which identifiers appeared on the left hand side of some previous assignment statement. This turns out to be so hard

that even designers of real programming languages omit enforcement of contextual rules from the grammar! In fact, while the official grammar for Java will not derive this program:

```
class A {2 / == {{;
```

and report a **syntax error**, a Java compiler will say that this program:

```
class A {int x = y;}
```

is structurally well formed according to the official syntax of the Java language! The compilation unit consists of a type declaration that is a class declaration whose body consists of a field declaration with a type, a name, and an initializing expression which is an identifier. But we know this program is not legal, since the identifier *y* has not been declared. It's not only Java that has grammars overspecifying things: this C program has multiple problems despite being well-formed according to the C grammar:

```
/*
 * This program is derivable by the official grammar for C.
 */
int main() {
    f(x)->p = sqrt(3.3);
    return "dog";
}
```

This is because it's really really really really really really really really hard and really really really really really really really really ugly to define contextual rules grammatically! Capturing an "all identifiers must be declared before use" constraint would require that a grammar rule for expressions not just generate *any* identifier, but only certain identifiers known from some outside context. Context! In general, capturing context in a grammar leads to grammars that:

- are incredibly difficult for humans to figure out,
- are nearly impossible for humans to read (they're ridiculously long and incredibly weird),
- look nothing at all like the way we would check these rules in a compiler,
- give rise to trees that are weird since expansion of nodes is based on sibling nodes, not just the parent, which makes semantics hard to define, and
- horribly expensive even for a computer to parse.

Try the following exercise if you're not convinced:

***Exercise**: Write grammars for:*

- $\{a^n b^n c^n \mid n \geq 0\}$
- $\{a^{2^n} \mid n \geq 0\}$
- $\{ww \mid w \in \{a, b\}*\}$

Hint 1: Grammars for these languages will have to have at least some rules with multiple symbols on the left hand side. That is because there exist proofs (not shown here) that these three languages are not context-free.

Hint 2: Having trouble? Answers can be found on my notes on Language Theory.

***Exercise**: Do some research to see if there are any known lower complexity bounds on the parsing of context-sensitive grammars.*

Sometimes there are constraints that we *can* (theoretically) capture in a context-free syntax, but that we might not want to. Try the following two exercises:

***Exercise**: Write a syntax rule for nonnegative integers less than* 128.

***Exercise**: Write a syntax rule for nonnegative integers less than* 2^{63}. *Is this busy work?*

Defining Contextual Constraints

So many programming language rules require context. Here's a small sampling from various languages:

- Identifiers must not be redeclared within a scope
- Identifiers may not be used in a place where they might not have been initialized
- Types of all subexpressions must be consistent in an expression
- The correct number of arguments must appear in a call
- Access modifiers (private, protected, public) must be honored
- All execution paths through a function must end in a return
- All possible arms of a match or switch or discriminated union must be covered
- All abstract methods must be implemented or declared abstract
- All declared local variables must be used
- All private methods in a class must be used

- ...and so many more

For our running example language, Astro, our constraints are:

1. The following identifiers are built-in:
    - `π`, a number
    - `sqrt`, a function of exactly one argument
    - `sin`, a function of exactly one argument
    - `cos`, a function of exactly one argument
    - `hypot`, a function of exactly two arguments
2. An identifier cannot be used in an expression unless it is one of the built-in identifiers or has been previously assigned to.
3. All function calls must accept the proper number of arguments.
4. The built-in identifiers cannot be assigned to.
5. Identifiers declared as functions can only be called, not used in any other context. Identifiers declared as numbers cannot be called.

That's informal prose. Can we capture the rules in a formal notations? Well, context sensitive grammars are too hard to write, too hard to understand, and too hard to parse, so we need other options. There are at least two:

- Use a completely new kind of grammatical formalism. Some of these do exist, including Attribute Grammars and Two-Level Grammars.

- Push contextual constraints into the world of semantics. Let the syntax be happy with undeclared identifiers appearing in expressions (as long as the expressions are otherwise well formed). With this approach, we say that using undeclared identifiers is "meaningless," you know, a semantic problem. And because the meaning (ahem, semantics) is computed before the program is run, we would call constraint violations **static semantic errors**.

Which take do you prefer? Slonneger and Kurtz take the former approach, writing:

> *The classification of such an error entails some controversy. Language implementers, such as compiler writers, say that such an infraction belongs to the **static semantics** of a language since it involves the meaning of symbols and can be determined statically, which means solely derived from the text of the program. We argue that static errors belong to the syntax, not the semantics, of a language. Consider a program in which we declare a constant:*
>
> ```
> const c = 5;
> ```

In the context of this declaration, the following assignment commands are erroneous for essentially the same reason: It makes no sense to assign an expression value to a constant.

```
5 := 66;
c := 66;
```

The error in the first command can be determined based on the context-free grammar (BNF) of the language, but the second is normally recognized as part of checking the context constraints. Our view is that both errors involve the incorrect formation of symbols in a command—that is, the syntax of the language. The basis of these syntactic restrictions is to avoid commands that are meaningless given the usual model of the language.

Though it may be difficult to draw the line accurately between syntax and semantics, we hold that issues normally dealt with from the static text should be called syntax, and those that involve a program's behavior during execution be called semantics. Therefore we consider syntax to have two components: the context-free syntax defined by a BNF specification and the context-sensitive syntax consisting of context conditions or constraints that legal programs must obey.

Indentation-Sensitive Syntax

In addition to the context issues that exist between a language's syntax and its static semantics, similar issues exist between the lexical syntax and phrase syntax in languages that are indentation-sensitive. Traditionally, lexical analysis proceeds as if the size of whitespace runs don't matter very much. But in languages like Python and Haskell, they do. A lot. We cannot just skip any whitespace between tokens the way we've done so above!

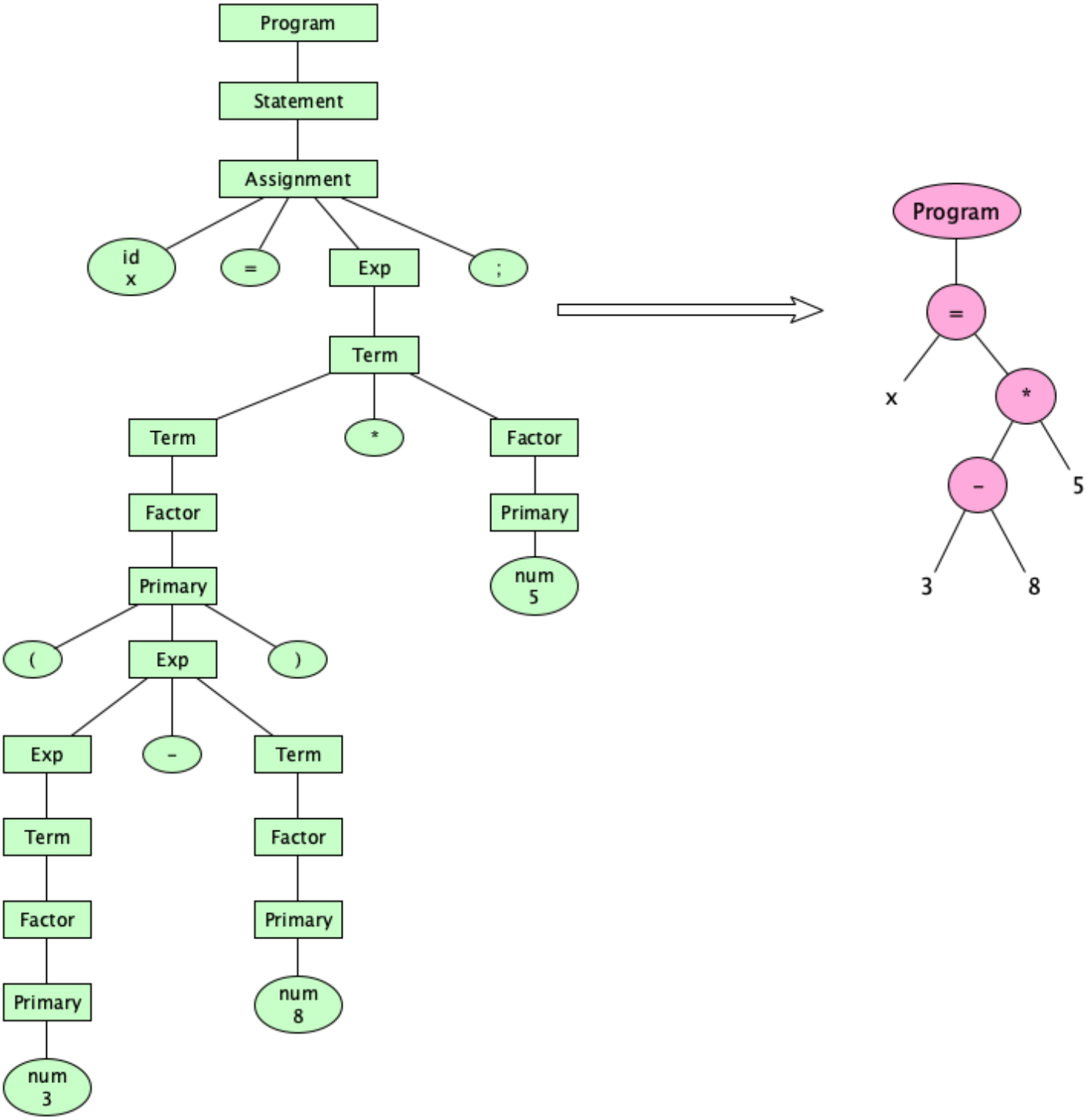
The way to handle indentation sensitivity is, during lexical analysis, keep track of indentation levels (in some sort of context object), and replace certain runs of whitespace with `INDENT` and `DEDENT` tokens.

The process isn't too hard. For complete details, read the section on indentation in the Lexical Analysis chapter of the Python Language Reference.

Abstract Syntax

Parsing is a lot of work. Specifying programs as strings of characters requires extra tokens (parentheses) and extra categories (Term, Factor, and Primary) in order to capture precedence, and crazy rule writing to get associativity correct. We have semicolons to terminate statements. Real languages have lots and lots of punctuation as well. Parse trees can get huge!

Dealing with spaces, comments, tokens, punctuation, precedence, and associativity is important in the early stages of language processing (the thing that interpreters, compilers, syntax highlighters, debuggers, and performance monitoring systems do). But once source code is parsed and the internal structure is uncovered, we can simplify the parse tree into something much simpler for the later stages of language processing (e.g., semantic analysis, optimization, translation, code generation, or execution). These later stages don't really *need* that much from a parse tree. We can abstract away a lot of the messy stuff, and translate our parse tree into an **abstract syntax tree**:



So much simpler! And by the way, since these simpler trees have become known as **abstract syntax trees** (ASTs), it turns out that, yes, parse trees have become known as **concrete syntax trees** (CSTs).

How do we come up with an abstract syntax? Focus on what a program is trying to express, without worrying about punctuation. You don't even have to worry about precedence and associativity (or even parentheses) because that stuff was already "taken care of" earlier. So cool! Here's how we think about it for Astro:

- A program is a sequence of statements
- A statement is either an assignment or a call

- An assignment is an identifier being assigned an expression
- A call has an identifier for the function name and a list of expressions for its arguments
- An expression can be a number, identifier, a unary operator applied to an operand, two expressions with a binary operator, or a call.

If you want to formalize this, you can write a **tree grammar**:

n :	Nml		
i :	Ide		
e :	Exp	$=$	$n \mid i \mid -e \mid e + e \mid e - e \mid e * e \mid e / e \mid e \backslash\% e \mid e ** e \mid i\ e^*$
s :	Stm	$=$	$i = e \mid \texttt{print}\ e$
p :	Pro	$=$	$\texttt{program}\ s^+$

Some people like to write tree grammars another way:

Program → Stmt+
Assign : Stmt → id Exp
Print : Stmt → Exp
num : Exp
id : Exp
Negate : Exp → Exp
Plus : Exp → Exp Exp
Minus : Exp → Exp Exp
Times : Exp → Exp Exp
Divide : Exp → Exp Exp
Remainder : Exp → Exp Exp
Power : Exp → Exp Exp
Call : Exp → id Exp*

When translating to an abstract syntax, we not only get rid of intermediate syntax categories, but we also drop parentheses, commas, semicolons, brackets, and pretty much all punctuation. Some of the smaller keywords that you see in larger programming languages (`in` , `to` , `of`) also are not needed. Punctuation really only exists to give a "tree structure" to our linear program text. Once we have uncovered the tree structure in the CST, we just carry it over naturally to the AST.

Don't break down the tokens either

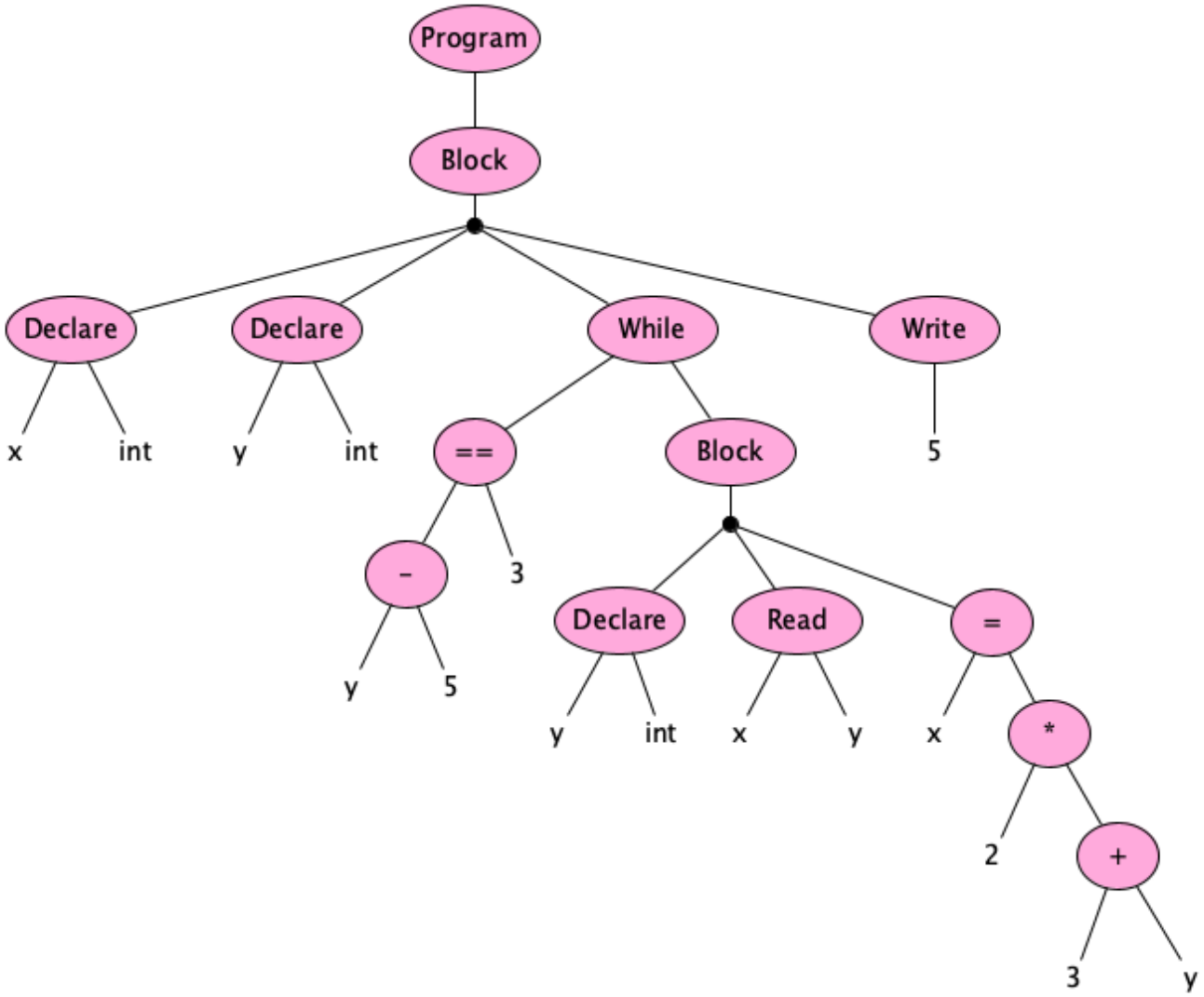
When writing an abstract syntax, you can assume that the tokens (the lexical syntax) is already done for you.All lexical categories (like identifiers and numeric literals in our example above) are primitive.

***Exercise**: Give the token stream and draw the concrete and abstract syntax trees for the Astro program* `print(3-5-8-2*8-9-7**3**1);`.

***Exercise**: Give the token stream and draw the concrete and abstract syntax trees for the Astro expression* `98 * (247 - 6) / 3 / 100 - 701`.

Multiple Concrete Syntax Trees

So ASTs allow us to focus on programming language capabilities without worrying about syntactic details that people can reasonably have different preferences about. Take a look at this AST (in some hypothetical language, not Astro):



CLASSWORK

Stop! Don't scroll by! Study this tree! What is this program "saying"?

It appears to be a program in a language whose tree grammar probably looks like this:

n :	Nml		
i :	Ide		
t :	Type	$=$	**int** | **bool**
e :	Exp	$=$	$n \mid i \mid -e \mid e+e \mid e-e \mid e*e \mid e/e \mid e**e$
			$\mid\ e<e \mid e\leq e \mid e==e \mid e\neq e \mid e>e \mid e\geq e \mid i\ e^*$
s :	Stmt	$=$	$i=e \mid$ **read** $i^+ \mid$ **write** $e^+ \mid$ **while** $e\ b$
d :	Dec	$=$	**declare** $i\ t$
b :	Block	$=$	$d^*\ s^+$
p :	Pro	$=$	**program** b

But what does the actual concrete syntax look like? There's no specific answer. There are an infinite number of possibilities! Here is just a small sampling of programs that could have generated that

AST:

```
program:
    var x, y: int
    // In this language, indentation defines blocks
    while y - 5 == 3:
        var y: int
        get(x)
        get(y)
        x = 2 * (3 + y)
    put(5)
```

```
int x, y;
while y - 5 = 3 {
    int y;
    STDIN -> x;
    STDIN -> y;
    x <- 2 * (3 + y);
}
STDOUT <- 5;
```

```
COMMENT THIS LOOKS LIKE OLD CODE
DECLARE INT X.
DECLARE INT Y.
WHILE DIFFERENCE OF Y AND 5 IS 3 LOOP:
    DECLARE INT Y.
    READ FROM STDIN INTO X.
    READ FROM STDIN INTO Y.
    MOVE PRODUCT OF 2 AND (SUM OF 3 AND Y) INTO X.
END LOOP.
WRITE 5 TO STDOUT.
```

```
(program
  (declare x int)
  (declare y int)
  (while (= (- y 5) 3)
    (define (y int))
    (read x y)
    (assign x (* 2 (+ 3 y)))
  )
```

```
  (write 5)
)
```

```
<program>
  <declare var="x" type="int"/>
  <declare var="y" type="int"/>
  <while>
    <equals>
      <minus><varexp ref="y"/><intlit value="5"/></minus>
      <intlit value="3"/>
    </equals>
    <declare var="y" type="int"/>
    <read vars="x y"/>
    <assign varexp="x">
      <times>
        <intlit value="2"/>
        <plus>
          <intlit value="3"/>
          <varexp ref="y"/>
        </plus>
      </times>
    </assign>
  </while>
  <write>
    <intlit value="5"/>
  </write>
</program>
```

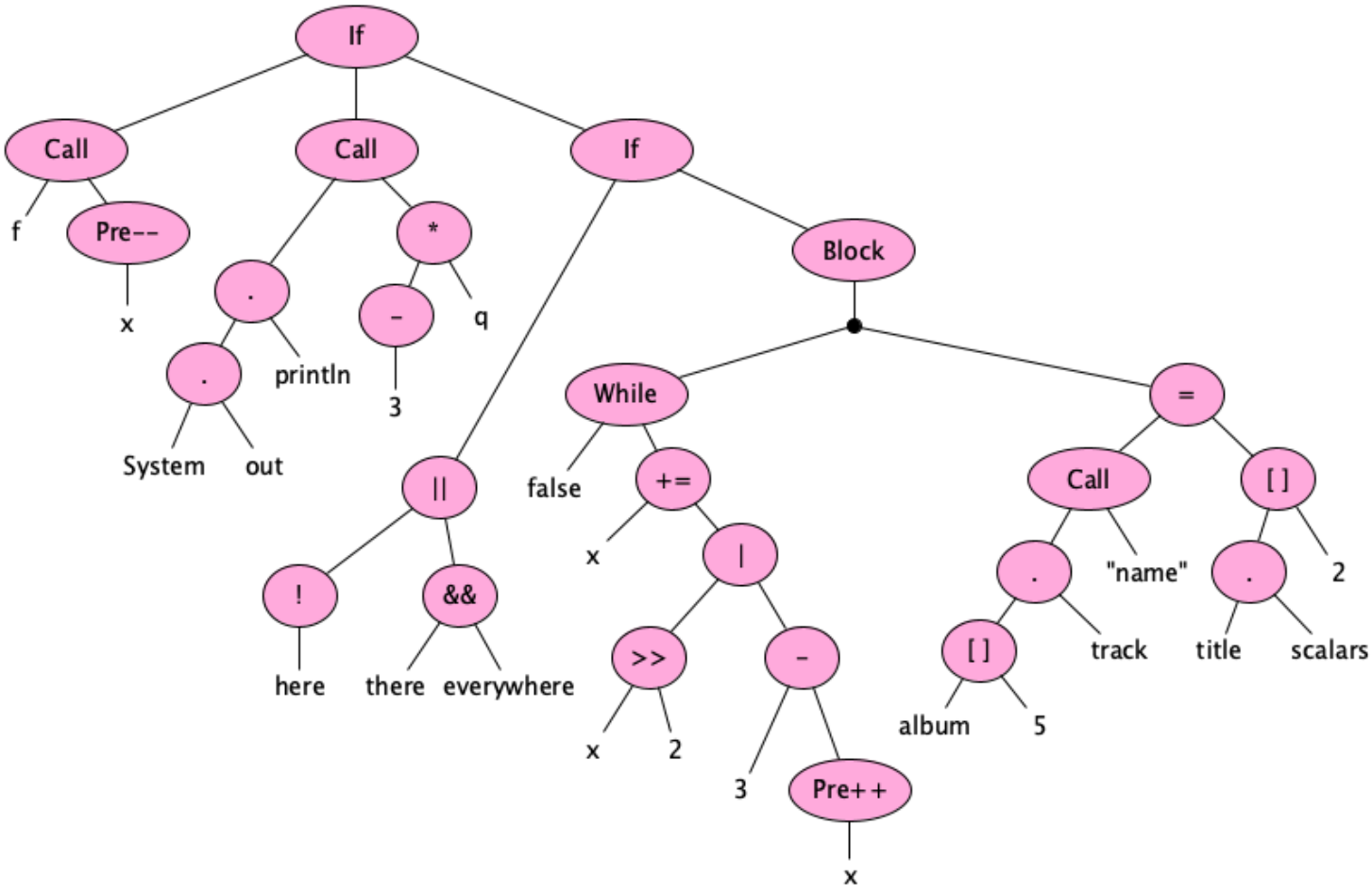
***Exercise**: Write the example in a JSON-like notation.*

***Exercise**: The fourth example above has an "S-expression syntax" which is seen in languages like Lisp and Clojure. In such languages, concrete and abstract syntaxes can be said to be very close to each other. Why is this claim true?*

More AST Examples

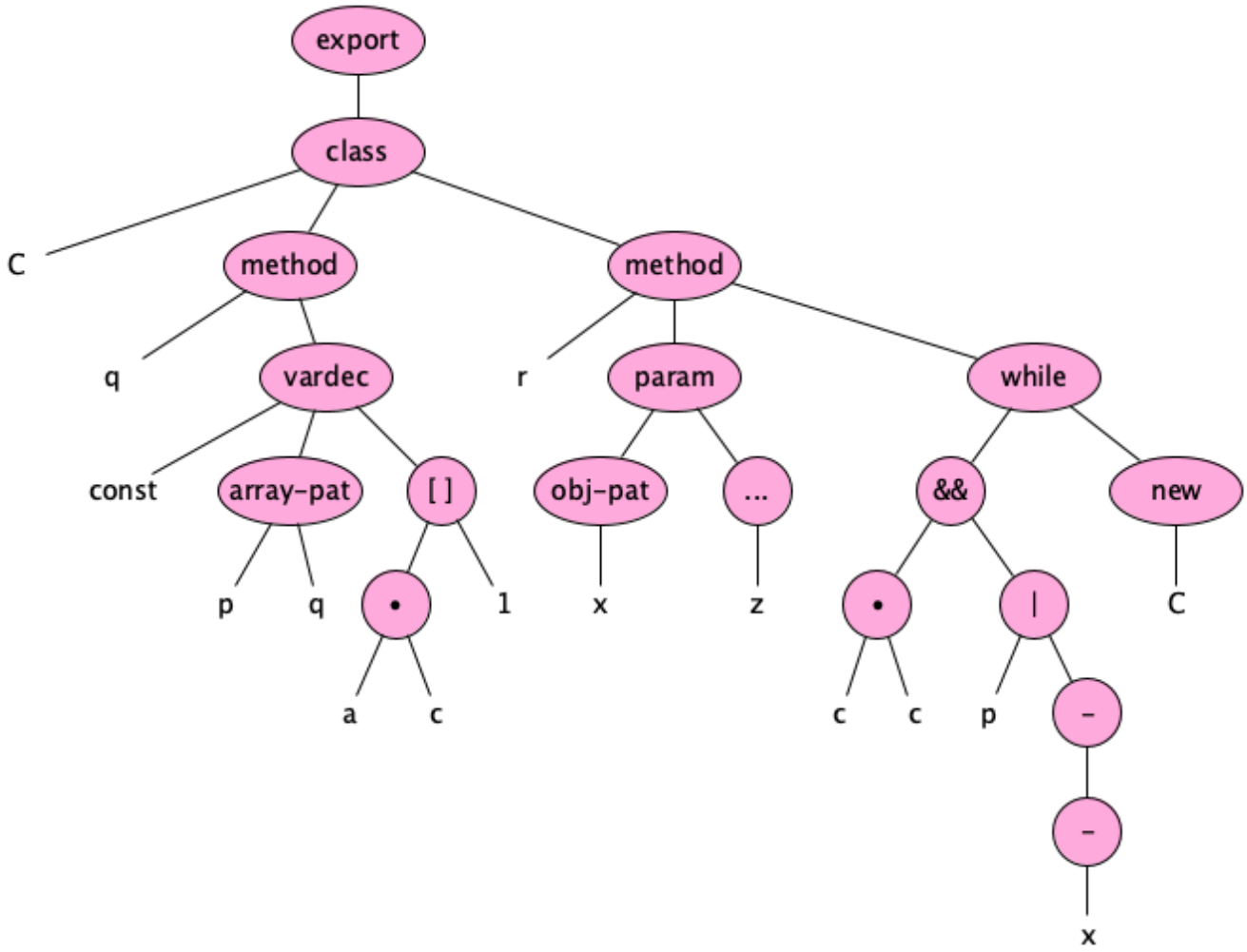
Getting a good feel for abstract syntax naturally takes practice. Here are a couple examples to study to get a feel for how abstract we need to get. Here's a fragment of Java:

```
if (((f(--x)))) {
    System.out.println(- 3*q);
} else if (!here||there    &&everywhere) {
    while (false) {
        x += x >> 2 | 3  -++  x;
    }
    album[5].track("name") = title.scalars[2];
}
```



And here's a little JavaScript program:

```
export class C {
  q() { const [p, q] = a.c[1]}
  r({x}, ...z) { while (c.c&&p|- -x) new C() }
}
```



Exploring Abstract Syntax

I've built an interactive application for you to explore JavaScript abstract syntax trees. Please try it out!. Hopefully you can get a sense of the connection between concrete and abstract syntax by doing so.

Syntax in the Real World

These notes were very much just an introduction to some of the theory surrounding syntax and syntax definitions. There is so much more to study. For example, you would do well to see how much work goes into writing a grammar for a real-world programming language. Here are a few to check out. Note the wide variety of notations:

- Python
- Java

- JavaScript
- Kotlin
- Go
- Swift
- C++
- C

> ***Exercise**: Browse each of these descriptions. Take notes that summarize the basic ideas of each. For each, how to they distinguish lexical from phrase syntax? How to they distinguish tokens from variables? What interesting conventions do they have that you had not thought of?*

> ***Exercise**: Find more official grammars for real languages, and send me links.*

Alternate Syntactic Descriptions

Hey, before we finish up, there's one last thing to cover.

So far, we've seen two approaches to syntax definition: Syntax Diagrams and a form of analytic grammar based strongly on Parsing Expression Grammars. It turns out there are dozens of published notations out there! You can use whatever you like, but there are some standardized forms that you should be aware of. The following four (CFG, BNF, EBNF, ABNF) are **historically significant** and very good to know. (You might even be expected to know these if you have a Computer Science degree.)

> ***These notations may not be as formal as you think!***
>
> *These traditional syntax notations generally do not make distinctions between the lexical and phrase syntaxes. You have to infer by context, or by side notes in a language definition, how tokens are formed and where spaces are allowed to go.*
>
> *This is why you should stick to analytic grammars!*

Context-Free Grammars

Chomsky discovered (or invented?) his Type-2 grammars, which he called Context-Free Grammars, as a minimalistic formalism for studying languages that do not have context. They are by design very "mathematical" and far too low-level to be used in real life. They just use individual symbols. They have no special operators for alternation or repetition. They are too hard to read and are never used directly in practice. A CFG is a 4-tuple (V, Σ, R, S), where V is the set of variables, Σ is the alphabet (of symbols in the language), $V \cap \Sigma = \varnothing$, $R \subseteq V \times (V \cup \Sigma)*$ is the rule set, and $S \in V$ is the start symbol. For the language of simple arithmetic expressions over natural numbers, a CFG is:

$$
\begin{(}
\quad \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, (,)\},
\quad \{E, T, F, N, D\},
\quad \{
\qquad (D, 0), (D, 1), (D, 2), (D, 3), (D, 4), (D, 5), (D, 6),
\qquad (D, 7), (D, 8), (D, 9), (N, D), (N, ND), (E, T), (E, E + T),
\qquad (E, E - T), (T, F), (T, T * F), (T, T/F), (F, N), (F, (E))
\quad \},
\quad E
\end{)}
$$

Rather than writing them in tuple form, we often just list the rules, making sure the start symbol appears first, and making the variables and alphabet symbols inferrable by context:

```
E → E + T
E → E - T
E → T
T → T * F
T → T / F
T → F
F → ( E )
F → N
N → N D
N → D
D → 0
D → 1
D → 2
D → 3
D → 4
D → 5
D → 6
D → 7
```

```
D → 8
D → 9
```

A pure CFG is a super-minimal notation great for studying properties of formal, made-up, simple, languages. But when defining the syntax of real languages we want more expressiveness! It's nice to have ways to specify things like keywords and spacing, which get super messy if you stick to the pure CFG notation.

BNF

With BNF, we add the ability to have our variables and alphabet symbols be written in a more readable form. To distinguish the two, we quote the *variables* with angle brackets. One special operator, `|` (meaning "or"), is permitted.

```
<expression> ::= <term>
               |  <expression> + <term>
               |  <expression> - <term>
<term>       ::= <factor>
               |  <term> * <factor>
               |  <term> / <factor>
<factor>     ::= <number>
               |  ( <expression> )
<number>     ::= <digit> | <number> <digit>
<digit>      ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Better than a CFG, but there's still not built-in way to do keywords and spaces.

EBNF

EBNF, or *Extended* Backus Naur Form, quotes the alphabet symbols instead of the variables, and adds operators for alternation (`|`), optionality (`?`), zero-or-more (`*`), and one-or-more (`+`).

```
EXP    = TERM (('+' | '-') TERM)*
TERM   = FACTOR (('*' | '/') FACTOR)*
FACTOR = NUMBER | '(' EXP ')'
NUMBER = ('0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9')+
```

Now we're getting more concise. Pure EBNF still doesn't have any mechanism for distinguishing lexical and phrase variables, so folks using EBNF will either write two grammars (one for the tokens, and one for the phrases), or use some other convention.

ABNF

ABNF, or *Augmented* Backus Naur Form, is the specification used to describe all the Internet Protocols. It features tools for repetition, alternatives, order-independence, and value ranges.

```
exp    = term *(("+" / "-") term)
term   = factor *(("*" / "/") factor)
factor = number / "(" exp ")"
number = %x30-39
```

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What are the utterances of a programming language called? HIDE ANSWER

Programs

What is syntax and how does it differ from semantics? HIDE ANSWER

Syntax is a specification of the structure of a language, while semantics specifies its meaning.

What are some spectra of syntactic forms in common programming languages? HIDE ANSWER

Pictures vs. text

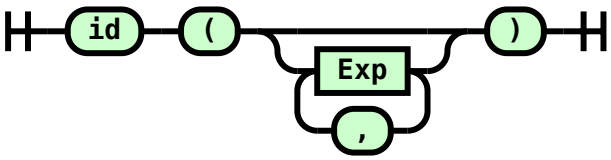
Symbols vs. words

Whitespace significance vs. insignificance

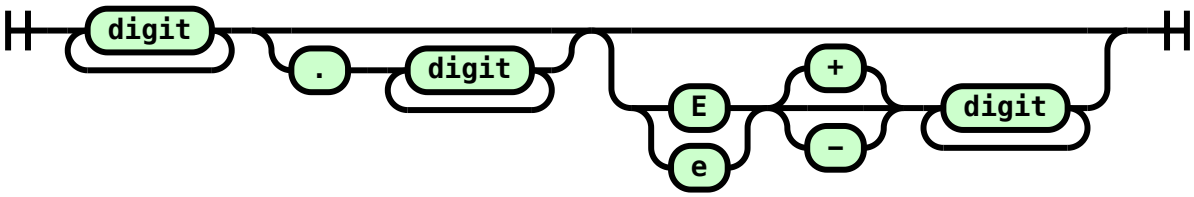
Indentation vs. curly braces vs. ends vs. parentheses

Prefix vs. infix vs. postfix

What does a syntax diagram for a function call look like, assuming the function being called must be a simple identifier? HIDE ANSWER



What does the syntax diagram for a typical floating-point literal look like? HIDE ANSWER



What are entities and identifiers? HIDE ANSWER

An entity is something that is manipulated by a program, such as a variable, literal, function, class, or module. An identifier is a name you can bind to an entity.

What is the difference between a statement and an expression? HIDE ANSWER

An expression is a construct that evaluates to a value, while a statement is a construct that performs an action.

What do we call words that look like identifiers but cannot be used as identifiers? HIDE ANSWER

Reserved words

What is a **reserved word**? HIDE ANSWER

A token that has the same lexical structure as an identifier but is not allowed to be used as one.

How do we, when defining a syntax, make sure reserved words cannot be used as identifiers?

HIDE ANSWER

We start the rule for identifiers with a negative lookahead for reserved words. Also, we need a negative lookahead after each reserved word to make sure it doesn't prohibit identifiers that may have a reserved word as a proper prefix.

What is the difference between the lexical and phrase syntax? HIDE ANSWER

The lexical syntax defines how individual characters are grouped into words, called tokens (such as identifiers, numerals, and punctuation); the phrase syntax groups words into higher level constructs (such as expressions and statements). These words can be separated by whitespace.

Why do grammars make a distinction between lexical and phrase syntaxes? Give two reasons.

HIDE ANSWER

1. To avoid littering nearly every rule with whitespace sequences.
2. To conceptually simplify the syntax, breaking it down into two manageable parts.

What convention are we using on this page to distinguish lexical and phrase variables?

HIDE ANSWER

Lexical variables begin with a lowercase letter; phrase categories begin with an uppercase letter.

What exactly is a **token**? HIDE ANSWER

A primitive element for a grammar's phrase structure, e.g., a keyword, identifier, numeric literal, boolean literal, simple string literal (without interpolation), or symbol.

What happens during **tokenization**? What kinds of language constructs are "eliminated" in this process? HIDE ANSWER

Characters in the source code are grouped into tokens. Comments and spaces between tokens are dropped. Spaces within string literals are kept of course.

Where do we generally cut off a parse tree? HIDE ANSWER

At the token level.

What does it mean for a grammar to be **ambiguous**? Answer in precise, technical terms. HIDE ANSWER

A grammar is ambiguous if there exists a string that has two distinct derivation trees in the grammar.

What kind of rule is a major source of ambiguity, one that is common for beginners to write? HIDE ANSWER

Rules with multiple components the same as the category being defined, e.g., $exp \longrightarrow exp\ op\ exp \mid \cdots$

How is operator precedence captured in a language definition? HIDE ANSWER

We use multiple syntactic categories for expressions; for example Expression, Term, Factor, Primary.

How do we capture the conventional precedences and associativities of binary arithmetic operators in an analytic grammar? HIDE ANSWER

$$
\begin{array}{lcl}
Exp & \longleftarrow & (Exp\ addop)?\ Term \\
Term & \longleftarrow & (Term\ mulop)?\ Factor \\
Factor & \longleftarrow & Primary\ (\texttt{"**"}\ Factor)?
\end{array}
$$

What is a good way to treat the expression `-3**2` in a language definition? HIDE ANSWER

It should be made illegal by the grammar! This eliminates the possible confusion about whether it means `(-3)**2` or `-(3**2)`, since it requires programmers to use parentheses.

What kinds of rules about program legality are not context free? Find as many as you can.

HIDE ANSWER

Some of them are:

- Type checking
- Redeclaration of identifiers within a scope
- Use of an undeclared identifier
- Matching arguments to parameters in a call
- Access checks (public, private, etc.)
- Identifiers must be used in all paths through their scope
- A return must appear in all paths through a function
- Pattern match exhaustiveness
- In a subclass, abstract methods must be implemented or declared abstract
- All private functions must be called within a module

There may be more.

What are two formalisms that have been created to capture context-sensitive rules?

HIDE ANSWER

Attribute Grammars and Two-Level Grammars

What do many language designers do in order to avoid the complexity of context-sensitive grammar rules? HIDE ANSWER

They push the rules into the world of semantics.

What kinds of rules about program legality can in principle be captured in a context-free syntax, but are generally not so captured in practice because the the required complexity of the rules?

HIDE ANSWER

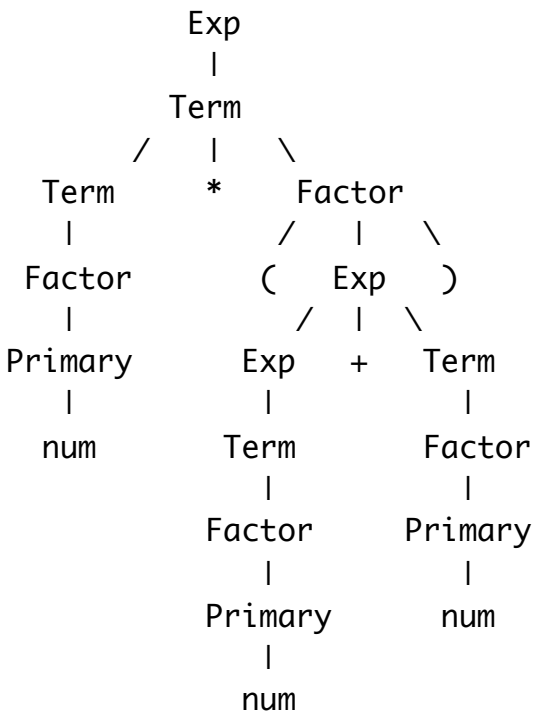
- Requiring `break` and `continue` in a loop (as this would require difference classes of statements)
- Requiring `return` in a function body (as this would require difference classes of statements)
- Enforcing ranges on certain numeric types.

In principle, can you capture the notion at all integer literals (in a hypothetical language) must be between -2147483648 and 2147483647? If so, why don't we? HIDE ANSWER

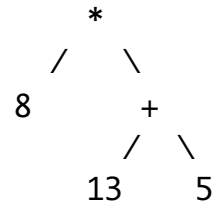
Yes, you certainly can, but the resulting specification is ugly af and really hard to decipher and understand.

Draw the parse tree and the abstract syntax tree for the expression `8 * (13 + 5)`, assuming a grammar with categories named Exp, Term, Factor, Primary, and num. HIDE ANSWER

The parse tree is:



The abstract syntax tree is:



Is indentation-sensitivity context-free? HIDE ANSWER

No

What is a concrete syntax? HIDE ANSWER

A precise specification of which strings (sequences of characters) are structurally legal programs.

What kinds of constructs appear in a concrete syntax but not in an abstract syntax? See how many items you can find. HIDE ANSWER

- Punctuation and delimiters
- Intermediate expression categories, e.g. Exp1, Exp2, Term, Factor, ...
- INDENT and DEDENT tokens

Given the expression `8 * (13  + 5)`, how many source code characters are there? When tokenized, how many tokens are there? How many nodes are there in the abstract syntax tree? HIDE ANSWER

14 characters, 7 tokens, 5 nodes.

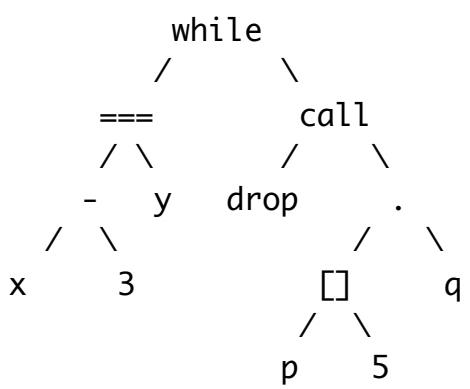
What kind of formal notation can capture an abstract syntax? HIDE ANSWER

A tree grammar.

What does the tree grammar for Astro look like? HIDE ANSWER

n : **Nml**
i : **Ide**
e : **Exp** $=$ $n \mid i \mid -e \mid e + e \mid e - e \mid e * e \mid e / e \mid e ** e \mid i\ e^*$
s : **Stm** $=$ $i = e \mid \mathbf{print}\ e$
p : **Pro** $=$ $\mathbf{program}\ s^+$

What does the AST for the JavaScript expression `while (x - 3 === y) { drop(p[5].q) }` look like? HIDE ANSWER



What are some notations that have been used to specify the syntax of programming languages?

HIDE ANSWER

BNF, EBNF, ABNF.

What syntax notation is used for official Internet standards? HIDE ANSWER

ABNF

Summary

We've covered:

- ✅ Motivation for the study of syntax
- ✅ Difference between syntax and semantics
- ✅ How to design a syntax
- ✅ Syntax diagrams
- ✅ Grammars
- ✅ Lexical vs. Phrase rules
- ✅ Ambiguity

- ✅ Precedence and associativity
- ✅ The problem of context
- ✅ Abstract syntax
- ✅ Tree grammars
- ✅ Syntax IRL
- ✅ Alternative syntax descriptions